

UNIwersytet Ekonomiczny w Krakowie

Kolegium Nauk o Zarządzaniu i Jakości
Instytut Informatyki, Rachunkowości i Controllingu
Kierunek: Informatyka Stosowana
Specjalność: Inżynieria Oprogramowania



Brunon Mikołaj Socha

Nr albumu: 233861

Projekt i implementacja aplikacji integracyjnej dla Krajowego Systemu e-Faktur

Praca licencjacka

Promotor
Prof. UEK dr hab. Wit Urban

Kraków 2026

Spis treści

Wstęp	5
1 Krajowy System e-Faktur i założenia integracji	6
1.1 Wprowadzenie	6
1.2 Tło prawne i gospodarcze	6
1.2.1 Luka VAT	7
1.2.2 Dyrektywy UE 2014/55 oraz UE 2025/516	8
1.2.3 Wyzwania dla małych i średnich przedsiębiorstw	8
1.2.4 Faktury ustrukturyzowane	9
1.3 Założenia pracy	9
1.4 Ogólna architektura oprogramowania	10
2 Wykorzystane technologie	11
2.1 Wprowadzenie	11
2.2 Język programowania Go	11
2.2.1 Geneza oraz użycie Go	11
2.2.2 Cechy Go	11
2.2.3 Model współbieżności Go	12
2.2.4 Uzasadnienie wyboru Go do wykonania projektu	12
2.2.5 Przykład użycia Go w projekcie	13
2.3 Baza danych SQLite	15
2.3.1 Opis SQLite oraz jego cech	15
2.3.2 Uzasadnienie wykorzystania SQLite w projekcie	15
2.3.3 Przykład wykorzystania SQLite w projekcie	15
2.4 Protokół HTTP oraz styl architektoniczny REST API	16
2.4.1 Opis ogólny protokołu HTTP	16
2.4.2 Styl architektoniczny REST API	17
2.4.3 Zastosowanie HTTP oraz REST w projekcie	17
2.5 Format JSON	17
2.5.1 Opis ogólny formatu JSON	17
2.5.2 Cechy formatu JSON	18

2.5.3	Przykładowe zastosowanie formatu JSON w aplikacji	18
2.6	Format XML oraz schemat XSD	20
2.6.1	XML	20
2.6.2	Schematy XSD	20
2.6.3	FA(3)	20
2.7	HTMX	21
2.7.1	Opis ogólny i cechy HTMX	21
2.7.2	Uzasadnienie użycia HTMX w projekcie oraz przykład zastosowania .	21
2.8	Tailwind CSS	23
2.8.1	Opis Tailwind CSS	23
2.8.2	Uzasadnienie użycia Tailwind CSS oraz przykład jego zastosowania w projekcie	23
2.9	Docker	24
2.9.1	Opis ogólny Dockera	24
2.9.2	Różnica między kontenerem i maszyną wirtualną	24
2.9.3	Zastosowanie Dockera w projekcie	24
2.10	Środowisko Krajowego Systemu e-Faktur	25
2.10.1	Krajowy System e-Faktur jako system zewnętrzny	25
2.10.2	Wymagania komunikacyjne	25
2.10.3	Wpływ środowiska KSeF na projekt	25
3	Projekt, architektura aplikacji i jej zastosowanie	27
3.1	Wprowadzenie	27
3.2	Architektura rozwiązania	27
3.2.1	Warstwa wejścia	29
3.2.2	Warstwa przetwarzania danych	29
3.2.3	Warstwa przechowywania danych	29
3.2.4	Warstwa przetwarzania w tle	31
3.2.5	Warstwa integracji z systemami zewnętrznymi	31
3.2.6	Warstwa wizualna oraz kontroli	31
3.2.7	Warstwa bezpieczeństwa	31
3.3	Przepływ przetwarzania faktury	32
3.3.1	Wejście	32
3.3.2	Walidacja otrzymanych danych	34
3.3.3	Utworzenie pliku w formacie XML	34
3.3.4	Walidacja danych w formacie XML	35
3.3.5	Zapis danych	35
3.3.6	Lokalna kolejka oraz wysyłka	35
3.3.7	Webhooki	35

3.3.8	Panel użytkownika	36
3.4	Komunikacja z KSeF	36
3.4.1	Uwierzytelnienie	36
3.4.2	Otwieranie i zamykanie sesji	37
3.4.3	Przesyłanie faktur	37
3.4.4	Sprawdzenie statusu i pobranie Urzędowego Poświadczenia Odbioru . .	37
3.5	Cykl życia aplikacji	37
3.5.1	Konfiguracja	38
3.5.2	Uruchomienie aplikacji	40
3.5.3	Działanie aplikacji	40
3.5.4	Bezpieczne zamykanie aplikacji	42
3.5.5	Wdrożenie kontenerowe	42
3.6	Przykładowy scenariusz użycia aplikacji	43
3.6.1	Środowisko i cel	43
3.6.2	Uruchomienie aplikacji w ramach scenariusza	43
3.6.3	Scenariusz pomyślnego przetworzenia faktury	43
3.6.4	Scenariusz odrzucenia faktury przez KSeF	47
3.7	Ograniczenia aplikacji	50
	Zakończenie	52
	Bibliografia	53
	Spis tabel	57
	Spis rysunków	58
	Spis listingów	59

Wstęp

Krajowy System e-Faktur, w skrócie KSeF, to platforma online pozwalająca na wystawianie, odbieranie oraz przechowywanie faktur ustrukturyzowanych, opracowana przez Ministerstwo Finansów (Ministerstwo Finansów, 2025d).

Celem pracy jest zaprojektowanie oraz utworzenie oprogramowania umożliwiającego bezpieczną, nieinwazyjną i minimalizującą koszty integrację z Krajowym Systemem e-Faktur, w szczególności dla małych i średnich przedsiębiorstw. Projekt zrealizowany został w języku Go, z wykorzystaniem innych technologii takich jak SQLite oraz HTMX i przybrał formę kodu, który może być kompilowany do pojedynczych plików wykonywalnych, kompatybilnych z systemami Windows, macOS oraz Linux.

Zakres pracy obejmuje:

- analizę wymagań technicznych oraz oficjalnej dokumentacji Krajowego Systemu e-Faktur,
- omówienie technologii wykorzystanych do utworzenia oprogramowania,
- zaprojektowanie architektury oprogramowania,
- implementację oprogramowania,
- testowanie jego działania,
- praktyczne wykorzystanie go w testowym scenariuszu oraz ocenę działania na podstawie wyników.

Niniejsza praca ustrukturyzowana jest według powyższej listy. W rozdziale pierwszym omówione są ogólne założenia projektu, środowisko Krajowego Systemu e-Faktur oraz ogólna architektura oprogramowania. W rozdziale drugim wytłumaczone są pojęcia wymagane do zrozumienia rozdziału trzeciego oraz omówione są technologie wykorzystane do realizacji projektu, takie jak język programowania czy formaty elektronicznej wymiany danych. Rozdział trzeci składa się z omówienia architektury utworzonej aplikacji, jej komponentów oraz funkcji, wraz z przedstawieniem przykładowych przeprowadzonych scenariuszy testowych.

Rozdział 1

Krajowy System e-Faktur i założenia integracji

1.1 Wprowadzenie

Obowiązek wystawiania faktur za pomocą platformy został stopniowo wprowadzony na przestrzeni pierwszej połowy 2026 roku (Ministerstwo Finansów, 2025d).

Pierwszy etap wdrożenia rozpoczął się 1 lutego 2026 roku, dla firm, których sprzedaż w roku 2024 przekroczyła wartość 200 milionów PLN, wliczając VAT (Ministerstwo Finansów, 2025c). Drugi, bardziej znaczący dla projektu realizowanego w tej pracy termin przypadł na 1 kwietnia 2026 (Ministerstwo Finansów, 2025c) - poczynając od tego dnia, obowiązek wykorzystywania narzędzia obowiązuje prawie wszystkich, z jednym głównym wyjątkiem, opisanym poniżej.

W związku z szeroką gamą zmian, które spowoduje wdrożenie systemu, Ministerstwo Finansów pozostawia możliwość wystawiania faktur poza platformą do 31 grudnia 2026 roku, o ile suma brutto tych faktur nie przekroczy wartości 10 000 PLN na przestrzeni rozliczanego miesiąca (Ministerstwo Finansów, 2025g). Dnia 1 stycznia 2027 roku, obowiązek korzystania z Krajowego Systemu e-Faktur dotyczył będzie prawie wszystkich przedsiębiorców.

1.2 Tło prawne i gospodarcze

KSeF wprowadzany jest w celu pomniejszenia luki VAT, wywiązania się ze zgodności z dyrektywą UE 2025/516 oraz ułatwienia procesu fakturowania (Council of the European Union, 2025; Ministerstwo Finansów, 2025d) - jednak wprowadzenie tego systemu może wiązać się z wieloma komplikacjami dla małych i średnich przedsiębiorstw, biorąc pod uwagę skalę zmian, które niesie ze sobą pełna cyfryzacja.

1.2.1 Luka VAT

Luka VAT to różnica między przewidywaną kwotą podatku VAT wynikającą z konsumpcji dóbr i usług a faktycznym przychodem skarbu państwa na jego podstawie (Mazur et al., 2019). W przypadku Polski, na 2024 rok, wynosiła ona 6,9 proc., czyli ok. 21,5 miliarda złotych (Ministerstwo Finansów, n.d.-a).

Tabela 1: Wielkość luki VAT w Polsce w latach 2004–2023

Rok	Luka VAT w %	Luka VAT w mld zł
2004	18,5%	15,0
2005	15,0%	13,4
2006	11,1%	10,8
2007	8,9%	9,6
2008	15,5%	18,8
2009	20,3%	25,3
2010	18,7%	25,2
2011	19,7%	30,0
2012	25,6%	40,1
2013	25,7%	40,4
2014	24,1%	38,8
2015	23,9%	39,6
2016	20,0%	33,7
2017	14,0%	25,2
2018	14,2%	28,4
2019	13,9%	29,4
2020	11,6%	24,5
2021	5,6%	13,4
2022	8,4%	20,5
2023	10,5%	—

Źródło: Opracowanie własne na podstawie raportu Ministerstwa Finansów (lata 2004–2017) (Mazur et al., 2019) oraz raportu Komisji Europejskiej dla Polski (lata 2018–2023) (European Commission et al., 2025).

Jak widać w tabeli 1, wystąpił znaczący spadek luki po roku 2016. Można wywnioskować, że przyczyniło się do tego wprowadzenie JPK, czyli inaczej Jednolitego Pliku Kontrolnego - dokumentu elektronicznego, w którym przedsiębiorcy informują Urząd Skarbowy na temat

transakcji dokonanych w ramach działalności gospodarczej, takich jak zakup czy sprzedaż towarów lub usług (Ministerstwo Finansów, n.d.-b). Obowiązek składania plików JPK_VAT, w odmianie V7M (miesięcznej) lub V7K (kwartalnej) (Ministerstwo Finansów, 2025b) nałożony został na wszystkich podatników VAT w trzech terminach, między połową 2016 roku a początkiem 2018 (Ministerstwo Finansów, n.d.-b).

Jednolite Pliki Kontrolne jednak opierają się w głównej mierze na uczciwości podatników - są to pliki XML, do których wnoszone są informacje na temat faktur wystawionych oraz odebranych w określonym okresie rozliczeniowym, jednak same faktury nie są załączane, tak więc prawidłowość raportu może być sprawdzona jedynie za pomocą audytu. Kolejnym problemem jest fakt, że informacja na temat transakcji dociera do Urzędu Skarbowego do 25 dni (Ministerstwo Finansów, 2025b) po zakończeniu okresu rozliczeniowego. Wprowadzenie Krajowego Systemu e-Faktur ma wyeliminować te problemy poprzez eliminację tradycyjnych faktur oraz wprowadzenie faktur ustrukturyzowanych, które będą mogły być monitorowane na bieżąco.

1.2.2 Dyrektywy UE 2014/55 oraz UE 2025/516

Dnia 26 maja 2014 roku weszła w życie dyrektywa Parlamentu Europejskiego 2014/55 w sprawie fakturowania elektronicznego w zamówieniach publicznych (European Parliament & Council of the European Union, 2014). W celu ułatwienia handlu wewnątrzunijnego, wprowadzono standard określający zasady dotyczące zawartości faktur elektronicznych. 11 marca 2025 ustanowiono zaś dyrektywę 2025/516, znaną również jako ViDA (VAT in the Digital Age, w skrócie ViDA), nakazującą pełne wdrożenie unijnego standardu, również w transakcjach B2B, do lipca 2030 roku (Council of the European Union, 2025). Wprowadzenie Krajowego Systemu e-Faktur, a wraz z nim faktur ustrukturyzowanych, można traktować jako krok w stronę wywiązania się z tego obowiązku.

1.2.3 Wyzwania dla małych i średnich przedsiębiorstw

Wprowadzone zmiany całkowicie zmieniają proces wystawiania oraz odbierania faktur. Na rynku pojawiają się już pierwsze rozwiązania integracji z systemem, jednak większość z nich nie jest dopasowana do wymogów małych i średnich przedsiębiorstw - wdrożenie systemu ERP przynosi wysokie koszty i jest wyposażone w narzędzia, które większości mniejszych przedsiębiorców nie przyniosą korzyści, a rozwiązania SaaS (Software as a Service) funkcjonują jako abonament, wymagają udziału osób trzecich oraz wymagają stałego połączenia z internetem. Na rynku brakuje niezłożonego rozwiązania, które jest w stanie sprostać wymaganiom mniejszych przedsiębiorców, pozwala na pominięcie pośredników i posiada mechanizmy ochronne w razie problemów z połączeniem internetowym lub przerwy serwisowej Ministerstwa Finansów.

1.2.4 Faktury ustrukturyzowane

Wprowadzony system przyjmuje jedynie faktury ustrukturyzowane - czyli w formacie XML (eXtensible Markup Language), o jednolitym układzie danych, umożliwiającym automatyczną analizę danych (Ministerstwo Finansów, 2025a). Faktury najpierw walidowane są wobec schematu (schema, w formacie XSD) (Ministerstwo Finansów, 2025a), po czym otrzymują unikalny numer identyfikacyjny nadany przez KSeF i przechowywane są na okres 10 lat od końca roku, w którym zostały wystawione (Ministerstwo Finansów, 2025d). Na dzień powstawania tej pracy obowiązujący schemat to FA(3) (Ministerstwo Finansów, 2025a), opublikowany 30 czerwca 2025 roku i obowiązujący od 1 lutego 2026 roku.

Faktury te różnią się od „tradycyjnych” faktur elektronicznych w formacie PDF przede wszystkim tym, że mogą być odczytywane maszynowo - choć tracą na czytelności dla człowieka. Dlatego również system pozwala na pobieranie wizualizacji tych faktur w wyżej wspomnianym formacie (Ministerstwo Finansów, 2025a).

1.3 Założenia pracy

Projekt realizowany jest zakładając scenariusz, w którym wiele mniejszych przedsiębiorców, którzy nie wpisują się w kategorie obejmowane warunkami towarzyszącymi terminowi pierwszemu lub drugiemu, zdecyduje się na rozwiązanie problemu poprzez zatrudnienie księgowego, gdy będzie to potrzebne - zleceńowo w przypadku przekroczenia kwoty obrotów 10 000 PLN brutto w rozliczanym miesiącu, lub na stałe od początku roku 2027 (Ministerstwo Finansów, 2025g).

Oprogramowanie tak więc kierowane jest wobec mniejszych przedsiębiorców, dla których integracja z Krajowym Systemem e-Faktur może wiązać się z komplikacjami oraz znaczącym podniesieniem kosztów - i na podstawie tego założenia zostaną przyjęte wszystkie inne.

Założenia:

- oprogramowanie kierowane jest wobec małych i średnich przedsiębiorstw,
- oprogramowanie może być dostosowane do wymagań użytkownika poprzez konfigurację,
- oprogramowanie nie wymaga nowoczesnego sprzętu komputerowego,
- oprogramowanie przewiduje możliwe zakłócenia w komunikacji z Krajowym Systemem e-Faktur - przez niedostępność serwisu lub problemy z połączeniem internetowym - oraz zapewnia odpowiednie mechanizmy bezpieczeństwa do radzenia sobie z nimi, z pominięciem trybów offline/offline²⁴,
- oprogramowanie nie ingeruje w treść faktur poza walidacją i normalizacją (przykładowo, wypełnienie pustych pól, które muszą zawierać wartości domyślne) i zapewnia, że dane przekazywane do Urzędu Skarbowego poprzez KSeF odzwierciedlają dane wprowadzone przez użytkownika,

- oprogramowanie pozwala na przeglądanie faktur poprzez panel użytkownika.

Najważniejszym założeniem jednak jest poprawność danych wprowadzanych przez użytkownika, a zatem również brak odpowiedzialności oprogramowania za ewentualne błędy, takie jak przykładowo niepoprawna stawka VAT - system nie waliduje danych pod względem prawidłowości prawnej, ani nie zastępuje pełnoprawnych narzędzi księgowych.

1.4 Ogólna architektura oprogramowania

Oprogramowanie, którego dotyczy ta praca, pełni rolę pośrednika między systemami stosowanymi przez przedsiębiorcę lub pracownika a Krajowym Systemem e-Faktur. Jego główną rolą jest przyjęcie faktur od użytkownika/z systemu zewnętrznego poprzez interfejs HTTP API, przekonwertowanie ich do formatu akceptowanego przez KSeF, wysyłka, monitorowanie procesu pod względem ewentualnych nieprawidłowości oraz wysłanie informacji zwrotnej o statusie faktury.

Na oprogramowanie składają się następujące elementy:

- interfejs wejściowy, przyjmujący dane bezpośrednio od użytkownika lub systemów zewnętrznych,
- oprogramowanie przetwarzające faktury i konwertujące je do formatu akceptowanego przez KSeF,
- oprogramowanie komunikujące się z Krajowym Systemem e-Faktur, odpowiedzialne za wysyłanie faktur oraz pobieranie informacji na temat ich stanu - m.in. Urzędowe Poświadczenia Odbioru,
- lokalna baza danych do przechowywania danych na temat statusu wysyłki poszczególnych faktur,
- „kolejka” lokalna, do przechowywania danych w przypadku przerwania komunikacji z internetem/braku dostępności KSeF,
- panel z interfejsem graficznym przeznaczonym dla użytkownika, informujący o statusie wysyłki oraz o napotkanych nieprawidłowościach.

Oprogramowanie przyjmuje faktury od użytkownika/systemu zewnętrznego oraz przetwarza dane, przygotowując je do wysyłki i dodając do lokalnej kolejki. Następnie, warstwa odpowiedzialna za komunikację z KSeF weryfikuje połączenie, wysyła dane i nasłuchuje odpowiedzi, zależnie od której aktualizuje status faktury. W przypadku niepowodzenia, oprogramowanie dokonuje dodatkowych prób i ostatecznie oznacza status wysyłki jako niepowodzenie. Jeśli przyjęte przez interfejs HTTP API żądanie zawierało adres, na który powinna zostać wysłana informacja zwrotna, aplikacja zwraca informacje dotyczące pomyślnego przetworzenia faktury lub ewentualnego niepowodzenia.

Rozdział 2

Wykorzystane technologie

2.1 Wprowadzenie

W tym rozdziale omówione zostaną technologie (języki programowania, protokoły, usługi) wykorzystane do realizacji projektu.

2.2 Język programowania Go

2.2.1 Geneza oraz użycie Go

Go to język programowania zaprojektowany przez trzech pracowników Google - Roba Pike'a, Kena Thompsona oraz Roberta Griesemera - charakteryzujący się prostotą składni, która przypomina język C, z wydajnością języków kompilowanych, takich jak C++ lub Rust (The Go Authors, n.d.). Udostępniony publicznie został w 2009 roku. Go jest obecnie popularne w projektach backendowych oraz chmurowych, wykorzystywane przez przykładowo Google, Dropbox, Uber, PayPal, Soundcloud, BBC, Docker, Kubernetes, American Express oraz Netflix.

2.2.2 Cechy Go

Go jest językiem statycznie typowanym, o typowaniu silnym - gdzie typ zmiennej (przykładowo, liczba całkowita, liczba zmiennoprzecinkowa) znany jest przed uruchomieniem programu, a każda zmienna ma odgórnie ustalony typ w kodzie (Google, n.d.-a).

Jest również językiem kompilowanym - kod nie jest na bieżąco interpretowany, jak w przypadku przykładowo Pythona, dzięki czemu większość błędów jest wykrywana zanim program zostanie uruchomiony. Zamiast tego tworzony jest w pełni przenośny, pojedynczy, uruchamialny plik binarny.

Go wyposażone jest w prostą, jednak wysoko efektywną bibliotekę standardową, co pozwala na minimalizację zewnętrznych zależności (Google, n.d.-b). Słynie również ze swojego modelu współbieżności, opartego na *goroutines* w przeciwieństwie do tradycyjnie

wykorzystywanych wątków (The Go Authors, n.d.). Go zachęca do jawnej obsługi błędów w przypadku każdej funkcji, która może zwrócić błąd - co ułatwia gwarancję poprawnego funkcjonowania budowanej aplikacji.

2.2.3 Model współbieżności Go

Go wspiera współbieżne wykonywanie zadań w ramach jednego programu, dzięki swojej alternatywie dla wątków - goroutines (dalej: gorutyny). Język wyposażony jest w swój własny program planujący, który przypisuje osobne „gorutyny” wątkom systemu operacyjnego - co pozwala na przypisanie kilku pojedynczemu wątkowi (The Go Authors, n.d.).

Typowy wątek wymaga stosu o wielkości jednego do dwóch megabajtów, w czasie gdy gorutyna potrzebuje zaledwie kilku kilobajtów, a jej stos rośnie zależnie od wymagań. Różnicą jest również to, w jaki sposób dochodzi do dzielenia się danymi pomiędzy wątkami. Tradycyjne wątki odczytują i nadpisują współdzieloną pamięć, w czasie gdy Go zachęca do używania tzw. kanałów, przez które przesyłane mogą być dane lub sygnały, ułatwiając omijanie zjawiska wyścigu, to jest błędów wywołanych pseudolosową kolejnością wykonywanych współbieżnych zadań.

Biblioteka standardowa Go posiada również funkcjonalność zwaną kontekstem, pozwalającą na wyznaczanie terminów oraz wysyłanie sygnałów między skoordynowanymi elementami programów.

2.2.4 Uzasadnienie wyboru Go do wykonania projektu

Go zostało wybrane przede wszystkim z uwagi na jego znajomość przez autora pracy - ale również prostotę składni i pracy z tym językiem. Kod jest przejrzysty i prosty w podziale na osobne elementy, a dzięki wymaganej kompilacji programu przy każdej próbie jego uruchomienia wykrywanie większości błędów odbywało się samoistnie. Biorąc pod uwagę założenia projektu, język umożliwił prostotę wdrożenia - końcowe pliki uruchamialne są na systemach Windows, Linux i macOS.

Dzięki swojej bibliotece standardowej, cały projekt korzysta zaledwie z trzech bezpośrednich zewnętrznych zależności, jak przedstawiono w tabeli 2.

Tabela 2: Biblioteki zewnętrzne wykorzystane w projekcie

Biblioteka	Zastosowanie	Rola w projekcie
github.com/goccy/go-yaml	konfiguracja	Odczyt i przetwarzanie pliku konfiguracyjnego <code>config.yaml</code> w formacie YAML (Goshima, n.d.).

Biblioteka	Zastosowanie	Rola w projekcie
github.com/lestrrat-go/helium v0.8.0	walidacja XML	Walidacja wygenerowanych faktur XML wobec oficjalnego schematu XSD faktur ustrukturyzowanych FA(3) (The Helium Authors, 2026).
modernc.org/sqlite	baza danych	Sterownik do bazy danych SQLite („sqlite package - modernc.org/sqlite - Go Packages,” n.d.).

Źródło: opracowanie własne.

Go, podobnie jak C, posiada wskaźniki pamięci, które wykorzystane zostały w projekcie do opcjonalnych pól faktur oraz odróżniania brakujących danych od danych, które są zwyczajnie puste. Najważniejszym elementem jednak są wspomniane w poprzedniej subsekcji gorutyny, umożliwiające aplikacji współbieżną obsługę żądań HTTP oraz cykliczne skanowanie bazy danych w poszukiwaniu rekordów do przetworzenia. Sama wysyłka faktur oraz webhooków (Red Hat, 2024) również jest współbieżna, a limit maksymalnych asynchronicznych zadań narzucany jest przez pojemność kanałów. Aplikacja oczekuje zakończenia obecnej próby przed rozpoczęciem kolejnego cyklu działania poprzez użycie `WaitGroup`, a użycie kontekstu pozwoliło na wprowadzenie mechanizmu bezpiecznego zamykania programu.

2.2.5 Przykład użycia Go w projekcie

```

1 var wg sync.WaitGroup
2 for _, inv := range invoices {
3     wg.Add(1)
4     go func(invoiceToProcess *models.Invoice) {
5         defer wg.Done()
6         c <- struct{}{}
7         defer func() {
8             <-c
9         }()
10        app.processInvoice(invoiceToProcess, inSession)
11    }(inv)
12 }
13 wg.Wait()

```

Listing 1: Przykład współbieżnego przetwarzania faktur

W powyższym fragmencie 1 przedstawiono, w jaki sposób aplikacja korzysta z modelu współbieżności języka Go w trakcie przetwarzania faktur. Tworzona jest `WaitGroup` pozwalająca na koordynację gorutyn. Następnie, dla każdej z zebranych faktur powstaje osobna gorutyna,

która zajmuje pojedyncze miejsce w kanale o limicie ustalonym przez użytkownika w pliku konfiguracyjnym. Po wywołaniu i zakończeniu działania funkcji `app.processInvoice()`, miejsce w kanale jest zwalniane, a funkcja zmniejsza licznik prac `WaitGroup`. Kanał przyjmuje pusty obiekt `struct{}`, z uwagi na to, że w przeciwieństwie do innych typów danych zajmuje zero pamięci i nie przechowuje żadnych informacji, tak więc wystarcza do spełnienia roli semafora (Google, n.d.-a). Aplikacja następnie oczekuje, aż wszystkie gorutyny zakończą pracę.

```
1 ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.
    SIGTERM)
2 defer stop()
3 shutdownC := make(chan struct{}, 1)
4 go app.startSender(ctx, shutdownC)
5
6 <-ctx.Done()
7 app.infoLog.Printf("event=shutdown_started")
8 shutdownCtx, cancel := context.WithTimeout(
9     context.Background(),
10    time.Duration(app.config.ShutdownTimeoutSec)*time.Second,
11 )
12 defer cancel()
13
14 if err := srv.Shutdown(shutdownCtx); err != nil {
15     app.errorLog.Fatalf("event=shutdown_failed error=%q", err.Error())
16 }
17
18 <-shutdownC
19 app.infoLog.Printf("event=shutdown_finished")
```

Listing 2: Przykład mechanizmu bezpiecznego zamykania aplikacji

W fragmencie 2 zaprezentowany został mechanizm bezpiecznego zamykania aplikacji. Tworzony jest kontekst oczekujący sygnału zamknięcia `SIGINT` (zatrzymania) lub `SIGTERM` (terminacji procesu). Powstaje również buforowany kanał pozwalający na przyjęcie lub otrzymanie pojedynczego pustego obiektu `struct{}`. Wywoływany jest główny mechanizm przetwarzania faktur poprzez uruchomienie gorutyny z funkcją `app.startSender()`, której przekazywany jest kanał. Gdy kontekst otrzyma sygnał zamknięcia, gorutyna informowana jest, że ma zakończyć pracę, a aplikacja wyczekuje na odpowiedź na kanale. W tym samym czasie, serwer HTTP otrzymuje czas na zakończenie pracy, z limitem określonym przez użytkownika w pliku konfiguracyjnym. Gdy mechanizm przetwarzania faktur zakończy pracę, aplikacja zamknie się.

2.3 Baza danych SQLite

2.3.1 Opis SQLite oraz jego cech

SQLite powstało jako prosty silnik baz danych SQL w 2000 roku, utworzony przez Dwayne'a Richarda Hippa w trakcie pracy dla General Dynamics, przy kontrakcie z Marynarką Wojenną Stanów Zjednoczonych. Jest obecnie najpopularniejszym silnikiem baz danych na świecie, używanym przez telefony komórkowe, systemy operacyjne oraz przeglądarki. Słynie przede wszystkim z tego, że nie wymaga serwera do działania - komunikacja odbywa się bezpośrednio poprzez pisanie do pojedynczego, lokalnego pliku, przez bibliotekę napisaną w języku C. Pomimo swojej prostoty, SQLite wspiera język SQL oraz operacje transakcyjne (SQLite Development Team, n.d.-a).

2.3.2 Uzasadnienie wykorzystania SQLite w projekcie

Jednym z głównych założeń projektu jest prostota wdrożenia oraz minimalizm aplikacji. SQLite pozwala na przetrzymywanie danych w pojedynczym, lokalnym pliku gwarantując trwałość danych offline oraz nie wymaga osobnego serwera, minimalizując wymagania administracyjne dla małych przedsiębiorstw (SQLite Development Team, n.d.-a). Zapewnia również wszystkie elementy wymagane przez projekt, w tym wcześniej wspomniane transakcje, gwarantujące bezpieczeństwo danych w trakcie pisania do bazy oraz pełne wsparcie dla języka SQL, pozwalając na proste tworzenie, aktualizowanie oraz usuwanie rekordów. SQLite zostało wybrane ze względu na swoją prostotę oraz spełnienie wszystkich wymogów projektu - w czasie gdy inne silniki również spełniają wymogi, jednak są bardziej skomplikowane oraz wyposażone w funkcje niepotrzebne do działania tworzonej aplikacji.

2.3.3 Przykład wykorzystania SQLite w projekcie

```
1 func Connect(dbPath string, busyTimeoutMs int) (*sql.DB, error) {
2     dirPath := filepath.Dir(dbPath)
3     if _, err := os.Stat(dirPath); os.IsNotExist(err) {
4         if err := os.MkdirAll(dirPath, 0755); err != nil {
5             return nil, err
6         }
7     }
8
9     db, err := sql.Open("sqlite", dbPath)
10    if err != nil {
11        return nil, err
12    }
13    if err := db.Ping(); err != nil {
```

```

14     db.Close()
15     return nil, err
16 }
17
18 concurrencySafe := fmt.Sprintf(
19     "PRAGMA journal_mode = WAL; "+
20     "PRAGMA synchronous = NORMAL; "+
21     "PRAGMA busy_timeout = %d; "+
22     "PRAGMA foreign_keys = ON;",
23     busyTimeoutMs,
24 )
25 if _, err := db.Exec(concurrencySafe); err != nil {
26     return nil, err
27 }
28 return db, nil
29 }

```

Listing 3: Przykład połączenia z bazą SQLite

Połączenie z bazą danych realizowane jest przez pakiet z biblioteki standardowej języka Go `database/sql`, służący jako interfejs do baz danych, jednak do komunikacji z samą bazą danych SQLite wykorzystywany jest osobny, zewnętrzny pakiet, jako sterownik dla `database/sql` używany następnie przez funkcję `sql.Open("sqlite", path)`. Przy pierwszym uruchomieniu aplikacji, tworzony jest katalog, a SQLite tworzy plik zawierający bazę. Następnie ustawiane są „pragmy”, instrukcje usprawniające i zabezpieczające działania asynchroniczne na rekordach. Przykładowo, instrukcja `busy_timeout` określa, przez jaki czas baza danych może być zablokowana w trakcie pisania do niej bez uznania blokady za wystąpienie błędu (SQLite Development Team, n.d.-b).

2.4 Protokół HTTP oraz styl architektoniczny REST API

2.4.1 Opis ogólny protokołu HTTP

HTTP, lub Hypertext Transfer Protocol, to protokół wykorzystywany do wymiany danych między urządzeniami w sieci. Opiera się na modelu żądań i odpowiedzi - przykładowo, urządzenie wysyła do serwera żądanie o określoną informację i otrzymuje ją z powrotem. Żądania składają się z kilku elementów, których najważniejszym jest „metoda” lub czasownik, określający, jakiej odpowiedzi/działania spodziewa się klient. W projekcie wykorzystane są trzy: GET, POST oraz DELETE, gdzie pierwszy żąda danych, drugi służy do ich przesłania, a trzeci prosi o usunięcie określonego elementu (Fielding et al., 2022).

Kolejnym elementem są nagłówki, czyli pary kluczy i wartości, rozwijające szczegóły

oczekiwanej odpowiedzi oraz przekazujące informacje na temat klienta - przykładowo, przeglądarkę, z której klient korzysta, lub formatu, w którym informacja zwrotna powinna zostać udzielona.

Same odpowiedzi zawierają kody statusu, które określają, do czego doprowadziło przyjęte żądanie. Przykładowo może to być odpowiedź 200, oznaczająca poprawne przyjęcie, lub 404, informującą, że żądana treść nie mogła zostać znaleziona.

2.4.2 Styl architektoniczny REST API

REST to skrót od Representational State Transfer (pol. transfer stanu przez reprezentację), czyli stylu budowania serwisów HTTP oraz interfejsów programowania aplikacji (Fielding, 2000). Jednym z założeń jest wsparcie dla operacji CRUD („Create, Read, Update, Delete”, pol. tworzenia, odczytu, aktualizacji i usunięcia) na udostępnianych poprzez odpowiednie endpointy zasobach - przykładowo, endpoint `deleteinvoice?external_id=` służyć może do usunięcia rekordu w bazie danych, poprzez otrzymywanie żądań HTTP z metodą DELETE, wskazujących określony identyfikator faktury. Ważną cechą REST jest brak stanu - każde żądanie powinno być wystarczające samo w sobie do wykonania działania, bez polegania na informacjach ukrytych wewnątrz aplikacji.

2.4.3 Zastosowanie HTTP oraz REST w projekcie

Protokół HTTP został wybrany do wykorzystania w projekcie przede wszystkim ze względu na swoją uniwersalność i popularność - większość systemów komunikuje się ze sobą za jego pomocą, tak więc jest oczywistym wyborem dla maksymalizacji kompatybilności z jak największą ilością zewnętrznych aplikacji. Styl architektoniczny REST został wybrany dla swojej przejrzystości i przewidywalności.

Utworzone oprogramowanie spodziewa się przyjmowania faktur od zewnętrznych systemów w postaci żądań zawierających dane w formacie JSON oraz eksponuje dodatkowe endpointy pozwalające na sprawdzanie statusu wysyłki do Krajowego Systemu e-Faktur lub usuwanie faktur z bazy danych. Panel użytkownika również oparty jest na HTTP, przesyłającym HTML.

Wykorzystanie HTTP ułatwiło również testowanie oprogramowania, przykładowo przy użyciu narzędzia curl.

2.5 Format JSON

2.5.1 Opis ogólny formatu JSON

JSON (JavaScript Object Notation, pol. „Notacja Obiektowa JavaScript”) to format plików oraz przekazywania danych utworzony przez Douglasa Crockforda we wczesnych latach

dwutysięcznych, w celu zastąpienia wtyczek przeglądarkowych takich jak Flash. Jest obecnie jednym z najpopularniejszych formatów wykorzystywanych do elektronicznej wymiany danych.

Pomimo, że nazwa odwołuje się do języka JavaScript, JSON jest niezależny od języka - większość nowoczesnych języków programowania wyposażona jest w narzędzia do zapisu i odczytu danych w tym formacie (Bray, 2017).

2.5.2 Cechy formatu JSON

JSON, dzięki swojej strukturze, jest czytelny dla człowieka. Elementy przedstawione są jako listy par kluczy i wartości, gdzie wartości mogą być różnego typu danych - w tym innych, złożonych obiektów, takich jak listy. Wymaga mniejszej ilości znaków niż większość innych formatów wykorzystywanych do elektronicznej wymiany danych, jak przykładowo XML.

2.5.3 Przykładowe zastosowanie formatu JSON w aplikacji

```
1 {
2   "invoice_type": "VAT",
3   "invoice_number": "FV/2026",
4   "issue_date": "2026-08-13",
5   "currency": "PLN",
6   "total_amount": 123.00,
7   "seller": {
8     "nip": "1234567890",
9     "name": "Sprzedawca z o.o.",
10    "country_code": "PL",
11    "address_line_1": "ul. Krakowska 1",
12    "address_line_2": "31-000 Krakow"
13  },
14  "buyer": {
15    "nip": "0987654321",
16    "name": "Nabywca Sp. z o.o.",
17    "country_code": "PL",
18    "address_line_1": "ul. Warszawska 1",
19    "address_line_2": "00-001 Warszawa"
20  },
21  "items": [
22    {
23      "line_number": 1,
24      "name": "Produkt",
25      "quantity": 1,
26      "unit_price_net": 100.00,
```

```

27     "net_amount": 100.00,
28     "tax_rate": "23"
29   }
30 ],
31 "tax_breakdowns": [
32   {
33     "tax_rate": "23",
34     "net_amount": 100.00,
35     "tax_amount": 23.00
36   }
37 ],
38 "callback_url": "https://system_zewnetrzny.com/webhook"
39 }

```

Listing 4: Przykładowa faktura w formacie JSON odbierana od systemu zewnętrznego

W powyższym przykładzie prezentowana jest faktura przesyłana przez system zewnętrzny do oprogramowania utworzonego w ramach projektu. Pola na początku dokumentu zawierają podstawowe dane, takie jak typ faktury, jej identyfikator, czy data wystawienia. Poniżej osadzone są informacje o sprzedawcy oraz nabywcy. Na końcu pojawiają się dwie listy - pierwsza, `items` zawierająca wszystkie produkty/usługi, których dotyczy faktura oraz druga, `tax_breakdowns` podsumowanie ich opodatkowania. Dane są czytelne dla człowieka, a przykład powyżej zawiera wszystkie dane potrzebne do ich transformacji do formatu akceptowanego przez Krajowy System e-Faktur.

```

1 {
2   "event": "invoice.accepted",
3   "invoice_id": 12,
4   "external_id": "FV/2026",
5   "status": "SENT",
6   "ksef_reference_number": "12345678901234567890",
7   "submission_reference": "ref123456",
8   "error_message": null,
9   "timestamp": "2026-08-13T10:16:20Z"
10 }

```

Listing 5: Przykładowa informacja zwrotna (webhook) w formacie JSON przesyłana do systemu zewnętrznego po poprawnym przetworzeniu faktury

Przykład 5 przedstawia ładunek JSON przesyłany do systemu zewnętrznego po poprawnym przetworzeniu, wysłaniu oraz sprawdzeniu statusu faktury w Krajowym Systemie e-Faktur. Podawane są, między innymi, identyfikator faktury w KSeF, identyfikator przesłania oraz dokładna data i godzina zdarzenia.

2.6 Format XML oraz schemat XSD

2.6.1 XML

XML (Extensible Markup Language, pol. „Rozszerzalny Język Znaczników”) to język służący przede wszystkim do elektronicznej wymiany danych (World Wide Web Consortium, 2008). Pozwala na tworzenie ustrukturyzowanych dokumentów, niezależnych od platformy - wszystkie dane przechowywane są jako prosty tekst. Dzięki swojej hierarchicznej strukturze oraz znacznikach, jest prosty do odczytania i walidacji przez oprogramowanie. Pierwsza wersja powstała w 1998 roku, a ostatnia, powszechnie używana do dziś, w 2008. Format utworzony został przez World Wide Web Consortium, międzynarodową organizację definiującą standardy WWW. Jego wykorzystanie w projekcie zostało ogólnie narzucone przez wymogi Krajowego Systemu e-Faktur - każda faktura do niego przesyłana musi być ustrukturyzowanym dokumentem w formacie XML, zgodnym z oficjalnym schematem.

2.6.2 Schematy XSD

XSD (XML Schema Definition, pol. „Definicja Schematu XML”) to format pozwalający na określenie, w jaki sposób ustrukturyzowany powinien być określony dokument w formacie XML (World Wide Web Consortium, 2012). XSD również zostało utworzone przez World Wide Web Consortium i jest przez nie polecane - istnieją inne formaty o podobnych zastosowaniach.

Schematy definiują przede wszystkim kolejność, ilość i rodzaj znaczników, które dokument powinien zawierać oraz dopuszczalne wartości w polach. Zastosowanie tego formatu pozwala na automatyczną walidację faktur otrzymanych i przetransformowanych do formatu XML przez oprogramowanie utworzone w ramach projektu przed podjęciem prób wysyłki do Krajowego Systemu e-Faktur.

2.6.3 FA(3)

Faktury wysyłane do Krajowego Systemu e-Faktur muszą być zgodne z oficjalnym schematem, udostępnionym przez Ministerstwo Finansów 30 czerwca 2025 roku, o nazwie FA(3). Schemat ten dokładnie określa, w jaki sposób ustrukturyzowany powinien być przesyłany dokument, jakie dane powinien zawierać oraz jakie wartości są dopuszczalne (Ministerstwo Finansów, 2025a).

Utworzone w ramach projektu oprogramowanie przetwarza otrzymywane od zewnętrznego systemu faktury w postaci JSON do formatu XML zgodnego z FA(3), następnie waliduje je wobec schematu przed wysyłką w poszukiwaniu niezgodności.

2.7 HTMX

2.7.1 Opis ogólny i cechy HTMX

HTMX, zapisywane również jako htmx, to lekka biblioteka języka JavaScript, utworzona przez Carsona Grossa, rozszerzająca możliwości HTML (HyperText Markup Language) poprzez dodanie nowych atrybutów. Pozwala między innymi na interakcję z serwerami za pomocą HTTP bezpośrednio z plików HTML oraz odświeżanie elementów strony bez jej całkowitego przeładowania. Głównym założeniem HTMX jest maksymalizacja możliwości języka HTML bez wspierania się ręcznie pisanym kodem w JavaScript (Big Sky Software, n.d.).

Przykładowo, w czasie gdy standardowy HTML pozwala jedynie na przesyłanie żądań HTTP o metodach GET i POST, HTMX umożliwia przesyłanie wszystkich innych - przykładowo, żądanie o metodzie DELETE może zostać wykonane po naciśnięciu przycisku na stronie poprzez nadanie przyciskowi atrybutu `hx-delete`.

2.7.2 Uzasadnienie użycia HTMX w projekcie oraz przykład zastosowania

Jednym z głównych założeń tworzonego oprogramowania jest jego prostota - zarówno w obsłudze, jak i w kodzie źródłowym. W większości projektów, w przypadku tworzenia interfejsu dla użytkownika, wykorzystywane są ciężkie biblioteki JavaScript lub TypeScript, jak przykładowo React czy Vue.js. HTMX zostało wybrane dzięki swojej prostocie oraz możliwościom - żądania HTTP do serwera wysyłane są bezpośrednio poprzez oznaczenie elementów interfejsu odpowiednimi tagami, a dynamiczne działanie interfejsu (odświeżanie listy faktur, ładowanie szczegółowego widoku faktury, manualny reset licznika prób ponownych wysyłki webhooków) obsługiwane jest przez zastępowanie fragmentów HTML przez HTML zwracany ze specjalnych endpointów, których żądania/odpowiedzi przesyłane są z tagiem `HX-Request`.

```
1 func (app *application) getDashboardInvoice(w http.ResponseWriter, r *http.  
    Request) {  
2     idRaw := r.URL.Query().Get("id")  
3     id, err := strconv.ParseInt(idRaw, 10, 64)  
4     if err != nil {  
5         app.errorLog.Printf("event=dashboard_invoice_id_invalid id=%q error=%q",  
            idRaw, err.Error())  
6         http.Error(w, "identyfikator faktury jest nieprawidłowy.", http.  
            StatusBadRequest)  
7         return  
8     }  
9     invoice, err := app.invoices.GetInvoice(id)  
10    if err != nil {  
11        if errors.Is(err, sql.ErrNoRows) {
```

```

12         http.Error(w, fmt.Sprintf("nie znaleziono faktury o identyfikatorze
13             %d.", id), http.StatusNotFound)
14     } else {
15         app.errorLog.Printf("event=dashboard_invoice_load_failed invoice_id
16             =%d error=%q", id, err.Error())
17         http.Error(w, "nie udało się pobrać faktury.", http.
18             StatusInternalServerError)
19     }
20     return
21 }
22 app.renderer.render(w, "invoice-container", invoice)
23 }

```

Listing 6: Przykład endpointu zwracającego fragment HTML dla HTMX

```

1 <div
2     hx-get="/ui/invoice?id={{ .Id }}"
3     hx-trigger="refreshInvoice from:body, retryWebhook from:body"
4     hx-target="this"
5     hx-swap="outerHTML"
6     class="w-full min-w-0 bg-white rounded-lg shadow overflow-hidden"
7 >
8     {{ if eq (printf "%s" .Status) "FAILED" }}
9         <button
10             type="button"
11             hx-delete="/ui/deleteinvoice?id={{ .Id }}"
12             hx-target="#invoice-target"
13             hx-swap="innerHTML"
14             class="bg-red-600 text-white px-3 py-1 rounded text-sm cursor-pointer"
15         >
16             Usuń fakturę
17         </button>
18     {{ end }}
19 </div>

```

Listing 7: Przykład użycia atrybutów HTMX dla widoku szczegółów faktury

Powyższe dwa fragmenty obrazują endpoint, zwracający HTML na podstawie wzoru poprzez funkcję `app.renderer.render()` oraz HTML, w którym za pomocą atrybutu `hx-target` wskazywany jest element, który zastąpiony będzie danymi wysyłanymi w odpowiedzi na żądanie `/ui/invoice?id={{ .Id }}` wykonane za pomocą `hx-get`. Żądanie wysyłane jest, gdy zostaną spełnione warunki opisane poprzez atrybut `hx-trigger` - w tym wypadku, gdy dojdzie do wydarzenia `refreshInvoice` lub `retryWebhook`, wywoływanych gdziekolwiek

na stronie.

2.8 Tailwind CSS

2.8.1 Opis Tailwind CSS

Tailwind CSS to biblioteka CSS, Cascading Style Sheets, czyli języka do stylowania elementów HTML. W przeciwieństwie do bibliotek takich jak Bootstrap, które dostarczają użytkownikowi z góry definiowane klasy, jak przykładowo gotowe, ostylowane przyciski, Tailwind CSS pozwala na stylowanie za pomocą wielu różnych klas, którymi oznaczane są elementy. Przykładowo, przycisk ostylować można dodając po jednej klasie dla koloru, kształtu, czcionki oraz koloru tekstu (Tailwind Labs, n.d.).

Tailwind CSS pozwala na bezpośrednie stylowanie elementów i podobnie jak HTMX pozwala na nadawanie elementom funkcjonalności bezpośrednio.

2.8.2 Uzasadnienie użycia Tailwind CSS oraz przykład jego zastosowania w projekcie

Tailwind CSS pozwala na szybkie stylowanie elementów za pomocą pojedynczych klas. Biorąc pod uwagę użycie HTMX oraz prostotę interfejsu eksponowanego przez oprogramowanie, praca z tą biblioteką ułatwiła szybkie utworzenie funkcjonalnego i przejrzystego panelu bez tworzenia osobnego, dedykowanego pliku ze stylami, umożliwiając pracę jak najbliżej samego HTML.

```
1 <button
2   type="button"
3   hx-delete="/ui/deleteinvoice?id={{ .Id }}"
4   hx-target="#invoice-target"
5   hx-swap="innerHTML"
6   class="bg-red-600 text-white px-3 py-1 rounded text-sm cursor-pointer"
7 >
8   Usuń fakturę
9 </button>
```

Listing 8: Przykład użycia klas Tailwind CSS w przycisku panelu użytkownika

Powyższy przykład obrazuje zastosowanie Tailwind CSS w stylowaniu przycisku panelu użytkownika. Element został oznaczony siedmioma klasami: `bg-red-600` nadającą czerwone tło, `text-white` nadającą tekstowi biały kolor, `px-3` i `py-1` definiujące odstępy tekstu od krawędzi bocznych i wertykalnych, `rounded` nadającą przyciskowi zaokrąglone rogi, `text-sm` określającą rozmiar tekstu oraz `cursor-pointer` zmieniającą kursor na wskazujący możliwość przyciśnięcia. Fragment kodu pokazuje również dlaczego Tailwind CSS dobrze współpracuje z HTMX - tuż nad klasami stylującymi element widzimy atrybuty nadające mu funkcjonalność.

Dzięki temu większość definicji wyglądu i działania elementu znajduje się bezpośrednio we fragmencie kodu, który go zawiera.

2.9 Docker

2.9.1 Opis ogólny Dockera

Docker to platforma pozwalająca na tzw. konteneryzację aplikacji, czyli tworzenie pakietów zawierających oprogramowanie oraz wymagane przez nie zależności systemowe, w celu wdrożenia na jakimkolwiek urządzeniu obsługującym Dockera (Docker, Inc., n.d.-a).

Obrazy kontenerów zawierają pliki i zależności wymagane przez określoną aplikację. Do uruchomienia aplikacji potrzebny jest również tzw. Dockerfile, czyli plik określający, w jaki sposób obraz powinien być zbudowany.

2.9.2 Różnica między kontenerem i maszyną wirtualną

Maszyny wirtualne symulują całą maszynę - z wydzieloną pamięcią, procesorem, kartą sieciową, własnym systemem operacyjnym, itd. Dzięki temu, określony program może być uruchomiony na jakimkolwiek urządzeniu, które jest w stanie uruchomić maszynę wirtualną z systemem, którego wymaga oprogramowanie.

Kontenery Dockera wirtualizują środowisko wymagane do uruchomienia aplikacji na jakimkolwiek systemie operacyjnym, który wspiera Dockera. Sam kontener zawiera zależności wymagane do uruchomienia określonego programu. Dzięki temu kontenery są lżejsze niż maszyny wirtualne, które symulują całe urządzenie ze wszystkimi jego komponentami (Docker, Inc., n.d.-b).

2.9.3 Zastosowanie Dockera w projekcie

Tworzone oprogramowanie ma być uruchamialne na możliwie jak najszerszej gamie urządzeń i systemów operacyjnych. Docker umożliwił utworzenie obrazu zawierającego aplikację i wszystkie jej zależności, wraz z plikiem Dockerfile, który określa dokładnie w jaki sposób obraz powinien zostać zbudowany. Plik konfiguracyjny oraz baza danych, mogą zostać dostarczone spoza kontenera.

Efektem końcowym jest możliwość uruchomienia aplikacji za pomocą jednej komendy - `docker compose up --build`.

2.10 Środowisko Krajowego Systemu e-Faktur

2.10.1 Krajowy System e-Faktur jako system zewnętrzny

Krajowy System e-Faktur to platforma do tworzenia, przesyłania, odbierania oraz przechowywania faktur.

Końcowym celem tworzonego oprogramowania jest integracja z Krajowym Systemem e-Faktur, w celu pośredniczenia między nim a systemami zewnętrznymi. Aplikacja, poza podstawową walidacją otrzymywanych danych, nie decyduje czy faktura zostanie przyjęta przez KSeF - służy za ledwie do transformacji otrzymywanych faktur w formacie JSON do wymaganego formatu XML zgodnego z oficjalnym schematem XSD FA(3), próby podjęcia wysyłki, sprawdzenia statusu oraz ewentualnego poinformowania nadawcy faktury o sukcesie bądź porażce.

2.10.2 Wymagania komunikacyjne

Środowisko Krajowego Systemu e-Faktur to API z eksponowanymi endpointami HTTP, przyjmującymi odpowiednie ładunki w formacie JSON. Ich struktura opisana jest w oficjalnej dokumentacji, pod adresem <https://api-test.ksef.mf.gov.pl/docs/v2/> (Ministerstwo Finansów, n.d.-c).

Do komunikacji z KSeF wymagane jest uwierzytelnienie, na podstawie tokena pobieranego przez użytkownika z Panelu Podatnika. Do uwierzytelnienia wykorzystywane są algorytmy kryptograficzne oraz klucze udostępniane przez Ministerstwo Finansów (Ministerstwo Finansów, n.d.-c).

W celu przesłania faktury, musi ona zostać przetransformowana w dokument XML zgodny z oficjalnym schematem oraz załączona w odpowiednim polu w ładunku JSON przesyłanym do KSeF. Następnie, sprawdzany jest status wysyłki, na podstawie identyfikatora zwracanego przez KSeF. Pomyślne przesłanie faktury nie oznacza jej akceptacji, więc by móc określić fakturę jako zaakceptowaną, odebrane musi zostać Urzędowe Poświadczenie Odbioru (Ministerstwo Finansów, n.d.-c).

2.10.3 Wpływ środowiska KSeF na projekt

Biorąc pod uwagę wspomniane w poprzedniej podsekcji wymagania, aplikacja musi być zdolna do przyjmowania faktur w postaci JSON, transformowania ich do odpowiedniego formatu oraz wysyłki do Krajowego Systemu e-Faktur, po wykonaniu odpowiednich kroków w celu uwierzytelnienia. Faktury przechowywane są w lokalnej bazie danych w celu ochrony danych w przypadku prac technicznych bądź innych problemów z komunikacją.

Oprogramowanie sprawdza również status przesyłek, i oznacza faktury jako przesłane dopiero, gdy odbierze Urzędowe Poświadczenie Odbioru. Ponieważ zewnętrzne systemy

wysyłające do oprogramowania dane potrzebują informacji dotyczących ostatecznego rezultatu prób wysyłki, aplikacja wyposażona jest w mechanizm webhooków, tj. danych zwrotnych, wysyłanych jeśli w oryginalnym żądaniu zawierającym fakturę zamieszczony został adres, na który nadawca chciałby ową informację otrzymać.

Rozdział 3

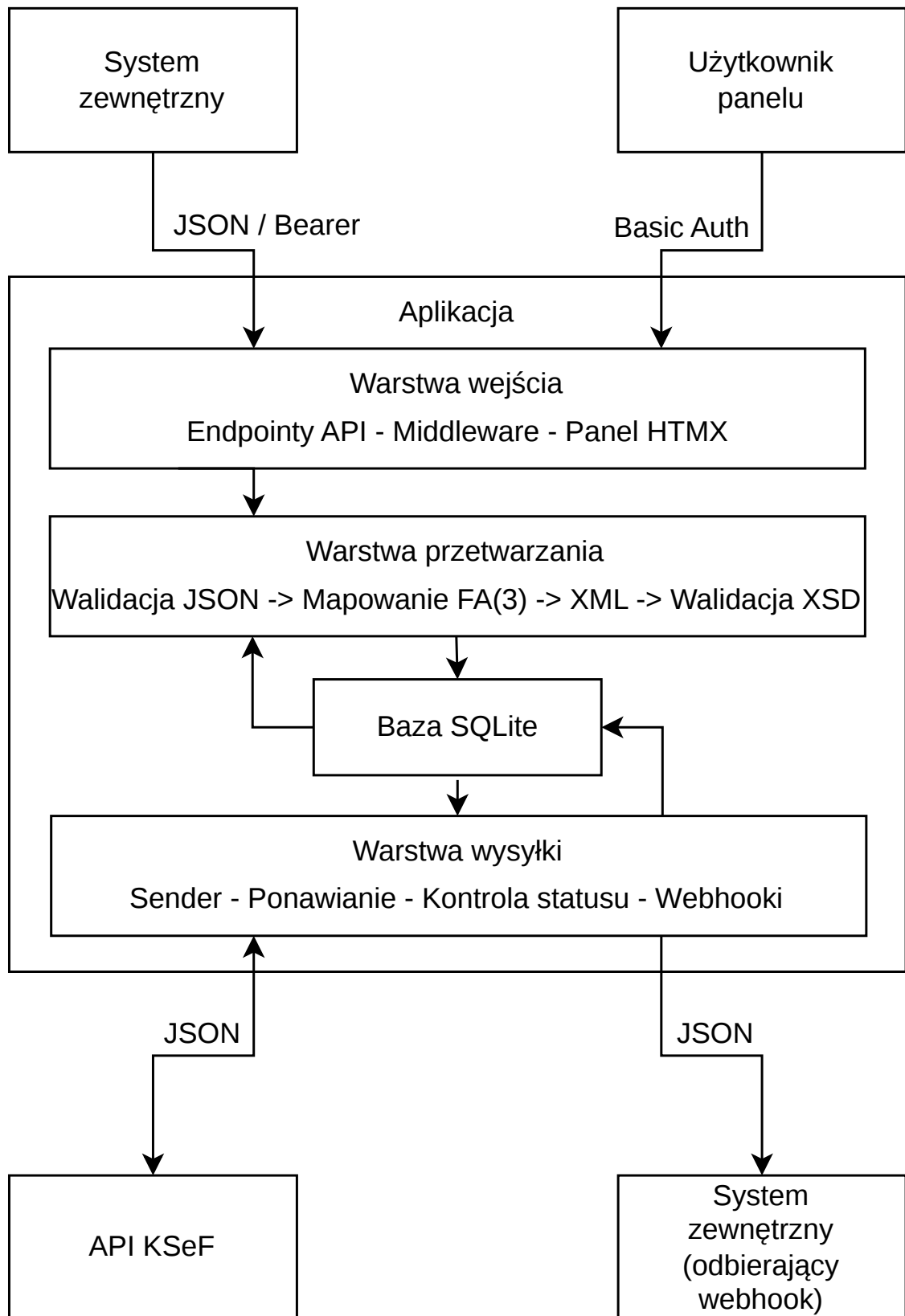
Projekt, architektura aplikacji i jej zastosowanie

3.1 Wprowadzenie

W celu dopasowania oprogramowania do założeń przedstawionych w poprzednim rozdziale, do realizacji wybrano narzędzia pozwalające na prostą integrację, możliwie najniższe wymagania sprzętowe, niską ilość zależności (ang. dependencies) oraz przejrzystość kodu źródłowego. W tym rozdziale przedstawiona zostanie ogólna architektura rozwiązania, jego elementy oraz przykładowe scenariusze testowe.

3.2 Architektura rozwiązania

Ogólną strukturę aplikacji oraz komunikację pomiędzy jej najważniejszymi elementami przedstawiono na rysunku 1:



Rysunek 1: Architektura aplikacji integracyjnej

Źródło: opracowanie własne.

3.2.1 Warstwa wejścia

Składa się z kolejnych endpointów serwera HTTP, przyjmujących ządania od zewnętrznych systemów w postaci JSON lub żądań od przeglądarki, zwracając HTML lub odpowiednie zdarzenia HTMX. Endpointy podzielone są na dwie kategorie - te odpowiedzialne za przyjmowanie faktur oraz zapytań z nimi związanych oraz obsługujące interfejs panelu użytkownika. Dane przyjęte w tej warstwie przekazywane są głębiej do aplikacji. Zwracane są ustrukturyzowane odpowiedzi w formacie JSON.

3.2.2 Warstwa przetwarzania danych

Przyjęte dane są procesowane poprzez sprawdzenie elementów faktury pod kątem poprawności. Otrzymany JSON transformowany jest w obiekty w języku Go, po czym transformowany w format XML. Walidator, utworzony przy uruchomieniu programu, sprawdza strukturę wygenerowanych danych, porównując ją do oficjalnego schematu XSD. Z pełnego kompletu wymaganych elementów tworzony jest obiekt odpowiadający rekordowi w bazie danych.

3.2.3 Warstwa przechowywania danych

Dane przechowywane są w relacyjnej bazie danych SQLite. Rozwiązanie to zostało wybrane ze względu na swoją prostotę w porównaniu do innych podobnych technologii (takich jak przykładowo PostgreSQL) oraz lokalne przechowywanie danych. Warstwa ta odpowiedzialna jest za odpowiedni zapis przyjmowanych faktur oraz ich obecnego stanu, korzystając z odpowiednich kolumn pozwalających innym elementom aplikacji wykonywać adekwatne działania na rekordach. Istnieje tylko pojedyncza tabela - `Invoices` (pol. Faktury) - która przechowuje dane przedstawione w tabeli 3.

Tabela 3: Najważniejsze kolumny tabeli `Invoices` w lokalnej bazie danych

Kolumna	Typ	Opis
<code>id</code>	INTEGER	Klucz główny rekordu.
<code>external_id</code>	TEXT	Numer faktury w systemie zewnętrznym.
<code>raw_json</code>	TEXT	Oryginalne dane otrzymane w formacie JSON.
<code>raw_xml</code>	TEXT	Faktura przekonwertowana do formatu XML.
<code>status</code>	TEXT	Status przetwarzania faktury - PENDING, PROCESSING, SENT, FAILED lub UNKNOWN.
<code>ksef_id</code>	TEXT	Numer faktury w KSeF.
<code>ksef_error</code>	TEXT	Błąd zwrócony przez KSeF w przypadku odrzucenia.
<code>upo_xml</code>	TEXT	Urzędowe Poświadczenie Odbioru w formacie XML.

Kolumna	Typ	Opis
attempt_count	INTEGER	Liczba prób ponownych wysłania faktury do KSeF.
created_at	DATETIME	Data utworzenia.
updated_at	DATETIME	Data ostatniej aktualizacji.
submission_reference	TEXT	Identyfikator przesłania, nadany przez KSeF.
callback_url	TEXT	Opcjonalny adres do przesłania informacji dotyczącej statusu faktury, po zakończeniu przetwarzania.
webhook_delivered	BINARY	Informacja, czy dane dotyczące statusu przesyłki zostały zwrócone.
webhook_attempt_count	INTEGER	Liczba prób ponownych dostarczenia informacji zwrotnej.
webhook_error	TEXT	Błąd w przypadku niepomyślnego dostarczenia informacji zwrotnej.

Źródło: opracowanie własne.

Kolumna status pozwala na pięć różnych wartości, przedstawionych w tabeli 4.

Tabela 4: Wartości kolumny status w tabeli Invoices

Status	Znaczenie
PENDING	Faktura została przyjęta przez aplikację i oczekuje na przetwarzanie oraz próbę wysyłki.
PROCESSING	Faktura została pobrana przez mechanizm wysyłki i obecnie jest w trakcie przetwarzania. Zależnie od efektu końcowego podejmowanych prób, status zostanie zmieniony.
SENT	Faktura została wysłana do Krajowego Systemu e-Faktur, a aplikacja otrzymała Urzędowe Poświadczenie Odbioru.
FAILED	Faktura nie mogła zostać wysłana lub została odrzucona przez Krajowy System e-Faktur. Podjęta została maksymalna liczba prób ponownych wysyłki, jeśli powód odrzucenia nie został podany przez KSeF.
UNKNOWN	Faktura została wysłana, ale nie otrzymano informacji zwrotnej z Krajowego Systemu e-Faktur. W trakcie kolejnych cykli działania aplikacji podjęte zostaną próby uzyskania odpowiedzi dotyczącej stanu faktury, by przypisać jej odpowiedni status.

Źródło: opracowanie własne.

W przypadku przyjęcia faktury o zewnętrznym identyfikatorze identycznym do faktury

już istniejącej w bazie danych, o statusie FAILED, rekord zostanie nadpisany, traktując nowe żądanie jako poprawioną wersję poprzedniej.

3.2.4 Warstwa przetwarzania w tle

Warstwa ta składa się z cyklicznego mechanizmu, który skanuje bazę danych w poszukiwaniu rekordów wymagających przetwarzania, zależnie od ich statusu. Mechanizm ten pobiera zbiór faktur (z limitem określanym przez użytkownika w pliku konfiguracyjnym), następnie uruchamia asynchroniczne procesy wysyłki dla każdego z nich. Faktury oraz webhooki, których wysyłka się nie powiodła, ale nie osiągnęły limitu prób ponownych wysyłki, również są procesowane w kolejnych cyklach. Pobierane są również faktury o statusie UNKNOWN w celu sprawdzenia ich statusu.

3.2.5 Warstwa integracji z systemami zewnętrznymi

Warstwa ta odpowiada za komunikację z Krajowym Systemem e-Faktur oraz systemami, od których aplikacja przyjmuje faktury. Autentykuje użytkownika na podstawie tokenu wprowadzonego w pliku ze zmiennymi środowiskowymi, odświeża pozyskany token pozwalający na wysyłkę w przypadku jego przedawnienia, otwiera i zamyka sesje interaktywne w celu wysyłki, sprawdza ich status oraz pobiera Urzędowe Poświadczenia Odbioru. Cała komunikacja odbywa się poprzez wysyłanie żądań w HTTP oraz przyjmowanie odpowiedzi w odpowiednich formatach opisanych przez oficjalną dokumentację Krajowego Systemu e-Faktur (Ministerstwo Finansów, n.d.-c).

Komunikacja odbywa się również z systemami, które wysyłając aplikacji fakturę, przesłały adres URL, na który wysłana ma zostać informacja zwrotna. Warstwa wysyła odpowiednio ustrukturyzowane żądania w formacie JSON zawierające informacje dotyczące statusu końcowego (SENT lub FAILED) faktur, które zostały przetworzone.

3.2.6 Warstwa wizualna oraz kontroli

Aplikacja eksponuje panel wizualny dla użytkownika, w postaci listy rekordów z bazy danych. Użytkownik może filtrować listę przez status faktury oraz wyszukiwać je za pomocą ich zewnętrznego identyfikatora. Możliwy jest również szczegółowy podgląd pojedynczej faktury oraz manualne jej usunięcie, jeśli jest o statusie FAILED. W przypadku, gdy wysyłki webhooka osiągnęły swój limit prób ponownych, możliwy jest również manualny reset licznika za pomocą przycisku. Panel jest automatycznie i dynamicznie odświeżany, bez przeładowywania strony.

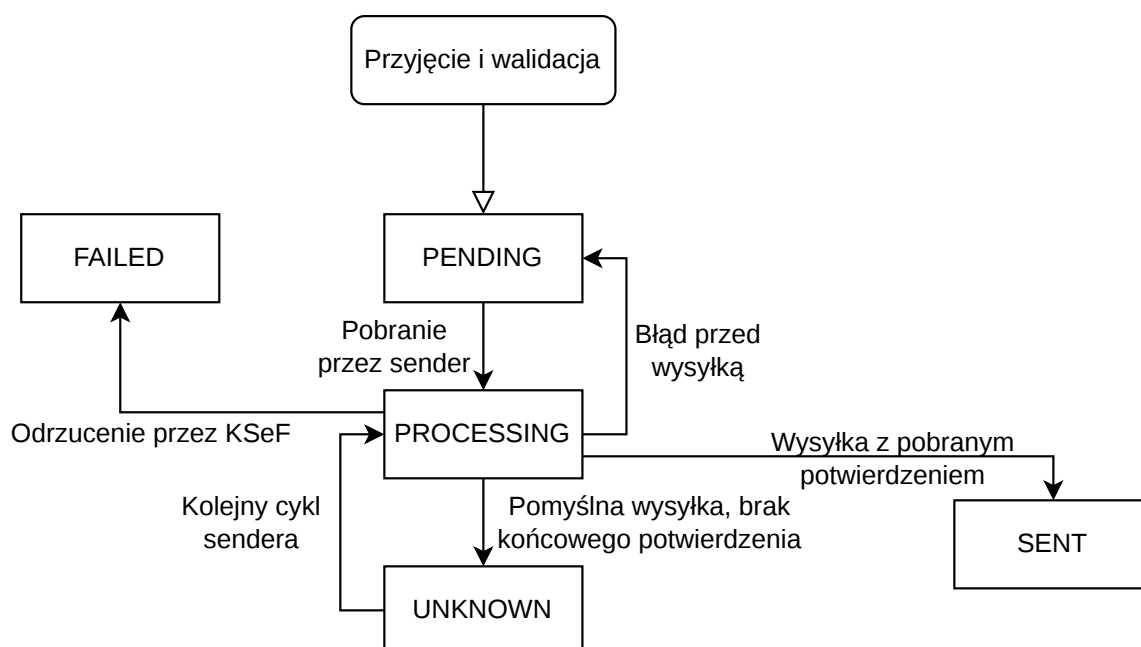
3.2.7 Warstwa bezpieczeństwa

Dane uwierzytelniające i poufne wymagane do działania aplikacji pobierane są ze zmiennych środowiskowych. APP_API_KEY to klucz, który musi być załączany do żądań przesyłanych przez

systemy zewnętrzne na endpointy służące do komunikacji z nimi, w nagłówku `Authorization` w schemacie `Bearer` (Jones & Hardt, 2012). `DASHBOARD_USERNAME` oraz `DASHBOARD_PASSWORD` chronią endpointy odpowiedzialne za interfejs i służą do uwierzytelnienia użytkownika przy próbie dostępu do wspomnianego w poprzedniej subsekcji panelu wizualnego, za pomocą HTTP Basic Authentication (Reschke, 2015). Niepowodzenie w uwierzytelnieniu w czasie interakcji z endpointami jego wymagającymi skutkuje odpowiedzią o kodzie HTTP 401, „Unauthorized”. `KSEF_TOKEN` to token uwierzytelniający, pobierany przez użytkownika z Panelu Podatnika Krajowego Systemu e-Faktur i umożliwia komunikację z samym KSeF.

3.3 Przepływ przetwarzania faktury

Możliwe zmiany statusu faktury podczas jej przetwarzania przedstawiono na rysunku 2.



Rysunek 2: Przejścia pomiędzy stanami przetwarzania faktury

Źródło: opracowanie własne.

3.3.1 Wejście

Proces rozpoczyna się poprzez przyjęcie danych w formacie JSON od systemu zewnętrznego poprzez żądanie POST do endpointu `/invoices`, eksponowanego przez aplikację. Format JSON został wybrany ze względu na swoją kompatybilność z HTTP API oraz szerokie wsparcie przez systemy zintegrowane i aplikacje backendowe. Jest on również czytelny dla człowieka. Aplikacja akceptuje dane w formacie JSON, służącym jako pośredni, integracyjny format, i używa ich do

stworzenia bardziej ustrukturyzowanego pliku XML dostosowanego do oficjalnego schematu FA(3) udostępnionego przez Ministerstwo Finansów (Ministerstwo Finansów, 2025a). Oprogramowanie spodziewa się danych przedstawionych w tabeli 5.

Tabela 5: Pola żądania wejściowego HTTP przekazywanego do endpointu /invoices

Pole	Wymagalność	Opis
invoice_type	TAK	Typ faktury - VAT, korygująca, zaliczkowa, rozliczeniowa, uproszczona, korygująca rozliczeniowa lub korygująca zaliczkowa.
invoice_number	TAK	Numer faktury w systemie zewnętrznym.
issue_date	TAK	Data wystawienia faktury w formacie RRRR-MM-DD.
currency	TAK	Kod waluty zgodny z ISO 4217 (International Organization for Standardization, n.d.).
exchange_rate	TAK / NIE	Kurs waluty dla walut innych niż PLN.
total_amount	TAK	Łączna wartość brutto faktury.
seller	TAK	Dane sprzedawcy - NIP, nazwa oraz adres.
buyer	TAK	Dane nabywcy - w przypadku faktury uproszczonej wymagany jest jedynie NIP (Ministerstwo Finansów, 2026).
items	TAK	Lista wierszy faktury, zawierająca nazwę, ilość, wartość netto oraz stawkę podatku dla każdej usługi/produktu.
tax_breakdowns	TAK	Wartości netto i podatku, podzielone przez stawki VAT.
payment	NIE	Dane dotyczące płatności - termin płatności, metoda i rachunek bankowy.
flags	NIE	Dodatkowe znaczniki, przykładowo płatność podzielona lub metoda kasowa.
callback_url	NIE	Adres URL, na który oprogramowanie wyśle informację o statusie przesyłki.
original_invoice_number	NIE	Numer oryginalnej faktury, wymagany w przypadku faktur korygujących.
original_issue_date	NIE	Data wystawienia oryginalnej faktury, wymagana w przypadku faktur korygujących.
original_ksef_id	NIE	Numer identyfikacyjny faktury w KSeF, wymagany w przypadku faktur korygujących.

Pole	Wymagalność	Opis
correction_reason	NIE	Powód korekty, wymagany w przypadku faktur korygujących.
corrected_buyer	TAK / NIE	Poprawione dane nabywcy, wymagane w przypadku faktur korygujących.

Źródło: opracowanie własne.

3.3.2 Walidacja otrzymanych danych

Faktura jest następnie przetwarzana - występuje walidacja podstawowych danych faktury w celu stwierdzenia, czy powinna zostać podjęta próba przesłania jej do Krajowego Systemu e-Faktur. Sprawdzane dane to:

- Typ faktury - faktura VAT, faktura korygująca, faktura zaliczkowa, faktura rozliczeniowa, faktura uproszczona, faktura korygująca rozliczeniowa lub faktura korygująca zaliczkowa.
- W przypadku faktur korygujących, sprawdzane są pola dotyczące danych korekty - powód, data wystawienia oryginalnej faktury, poprawność daty oraz poprawione dane kontrahenta. W przypadku faktur niekorygujących, pola sprawdzane są z założeniem, że powinny być puste.
- Podstawowa walidacja waluty - ilość znaków oraz w przypadku walut innych niż PLN, kurs między określoną walutą a PLN.
- Wiersze sprzedaży - aplikacja sprawdza, czy faktura nie jest pusta. Następnie waliduje każdy wiersz sprzedaży, poprzez sprawdzenie ilości określonego produktu/usługi, ceny na jednostkę oraz nałożenia odpowiedniego podatku. W przypadku faktur niekorygujących, odrzuca wartości netto poniżej zera. Wartości netto oraz podatku są zsumowane i walidowane wobec całkowitej wartości faktury.
- Dane kontrahentów - sprawdzane są Numery Identyfikacji Podatkowej obu stron wykonanej transakcji poprzez sprawdzenie ich odpowiednim algorytmem oraz nazwy i adresy stron. Walidowany jest również kod kraju sprzedawcy, a w przypadku jego braku, następuje wprowadzenie wartości domyślnej - PL. W przypadku faktury uproszczonej, dane kupującego, poza Numerem Identyfikacji Podatkowej, nie są wymagane (Ministerstwo Finansów, 2026).

3.3.3 Utworzenie pliku w formacie XML

Dane są następnie transformowane do formatu XML wymaganego przez Krajowy System e-Faktur, poprzez mapowanie odpowiednich pól JSON do pól wymaganych przez oficjalny schemat XSD,

m.in. nagłówek, informacje dotyczące sprzedawcy oraz nabywcy, wiersze sprzedaży, wartości podatkowe, informacje o płatności oraz ewentualnie pola korygujące (Ministerstwo Finansów, 2025a). Pola są generowane na podstawie struktur odpowiadających elementom końcowego pliku XML, a nie korzystając z manipulacji tekstem, minimalizując prawdopodobieństwo niepoprawności dokumentu.

3.3.4 Walidacja danych w formacie XML

Aplikacja waliduje nowo utworzone dane XML wobec schematu XSD udostępnionego przez Ministerstwo Finansów, sprawdzając poprawność pól pod względem typów i dopuszczalnych wartości oraz strukturę dokumentu - kolejność, odpowiednie zagnieżdżenie elementów (Ministerstwo Finansów, 2025a).

3.3.5 Zapis danych

W przypadku pomyślności poprzednich kroków, dane zapisywane są w lokalnej bazie danych SQLite, najpierw sprawdzając, czy identyfikator nadany fakturze przez zewnętrzny system już występuje we wcześniej zapisanych rekordach. Jeśli tak, a poprzednio zapisany rekord nie został poprawnie przesłany do Krajowego Systemu e-Faktur, aplikacja zastępuje ją, traktując nowe żądanie jako korektę poprzedniej faktury.

Tabela przedstawiająca strukturę rekordów przechowywanych w bazie danych została przedstawiona w tabeli 3.

3.3.6 Lokalna kolejka oraz wysyłka

Aplikacja skanuje bazę danych w poszukiwaniu faktur oznaczonych jako niewysłane oraz dodaje je do lokalnej kolejki. Jeśli kolejka nie jest pusta, otwierana jest sesja interaktywna przez uwierzytelnienie w KSeF, faktury są wysyłane oraz monitorowane pod względem statusu wysyłki. W przypadku akceptacji, faktura oznaczana jest jako wysłana - w przypadku nieznanego statusu, aplikacja ponowi zapytanie dotyczące faktury. Jeśli zaś dojdzie do porażki, aplikacja spróbuje wysłać fakturę ponownie w następnym cyklu działania, do momentu gdy nie zostanie osiągnięty określony przez użytkownika w pliku konfiguracyjnym limit prób ponownych. Wyjątkiem jest tutaj błąd połączenia z internetem - w tym wypadku, faktura pozostanie w kolejce, do momentu, gdy możliwa będzie wysyłka.

3.3.7 Webhooki

Jeśli w oryginalnych danych przekazanych aplikacji żądaniem figurował adres, na który wysłana powinna zostać informacja zwrotna dotycząca statusu wysyłki, mechanizm webhooków (*webhook* - dosł. hak sieciowy - aplikacja zapamiętuje dostawcę danych, na potrzebę przyszłej interakcji)

(Red Hat, 2024) prześle zewnętrznemu systemowi dane w formacie JSON po oznaczeniu faktury jako wysłana lub odrzucona, potwierdzając pomyślność operacji lub podając powód odrzucenia faktury. W przypadku błędu w trakcie próby przekazania danych, aplikacja podejmie kolejne próby, aż do osiągnięcia określonego przez użytkownika limitu prób ponownych. Zresetowanie licznika prób ponownych może zostać wykonane przez użytkownika z poziomu panelu dostępnego przez przeglądarkę.

Informacja zwrotna przesyłana jest w postaci przedstawionej w tabeli 6.

Tabela 6: Pola żądania HTTP z informacją zwrotną przesyłanego do systemu zewnętrznego

Pole	Typ	Opis
event	TEXT	Typ zdarzenia, przykładowo invoice.accepted lub invoice.failed.
invoice_id	INTEGER	Identyfikator faktury w aplikacji - klucz główny w bazie danych.
external_id	TEXT	Numer faktury pochodzący z systemu zewnętrznego.
status	TEXT	Status wysyłki - SENT lub FAILED.
ksef_reference_number	TEXT / NULL	Numer faktury w KSeF, jeśli faktura została przyjęta.
submission_reference	TEXT / NULL	Identyfikator przesłania, nadany przez KSeF.
error_message	TEXT / NULL	Błąd, który nastąpił w przypadku niepowodzenia.
timestamp	DATETIME	Czas przesyłki informacji zwrotnej.

Źródło: opracowanie własne.

3.3.8 Panel użytkownika

Aplikacja eksponuje również prosty panel na endpointzie `/ui/invoices` umożliwiający wgląd do historii wysyłek, filtrowanie faktur według statusu, wyszukiwanie pojedynczych rekordów oraz ich manualne usuwanie w przypadku niepomyślnej wysyłki i osiągnięcia limitu prób ponownych. Możliwy jest również reset licznika prób ponownych wysyłki webhooków, jeśli limit został poprzednio osiągnięty. Panel automatycznie odświeża swoją zawartość. Panel wykonany został z użyciem HTMX oraz ostylowany za pomocą Tailwind CSS.

3.4 Komunikacja z KSeF

Podstawowym elementem działania aplikacji jest interakcja z samym Krajowym Systemem e-Faktur - uwierzytelnienie, otwieranie i zamykanie sesji interaktywnych w celu wysyłki faktur

oraz sprawdzanie statusu i odbiór Urzędowych Poświadczeń Odbioru.

3.4.1 Uwierzytelnienie

Uwierzytelnienie następuje za każdym razem, gdy aplikacja musi wykonać akcję, która go wymaga. Aplikacja wykonuje puste żądanie POST do endpointu `/auth/challenge` eksponowanego przez KSeF, w odpowiedzi uzyskując „wyzwanie” - timestamp, który po połączeniu z pozyskanym wcześniej przez użytkownika tokenem API z panelu KSeF jest szyfrowany z użyciem klucza publicznego automatycznie pobieranego przez aplikację. Przesyłane następnie jest żądanie do endpointu `/auth/ksef-token`, zwracające tymczasowy token dostępu. Następnie, token ten wysyłany jest do `/auth/token/redeem`, pozyskując token dostępu oraz drugi, pozwalający na odświeżenie go. Oprogramowanie automatycznie sprawdza, kiedy dojdzie do przedawnienia, i automatycznie odświeży status uwierzytelnienia, jeśli jest taka potrzeba (Ministerstwo Finansów, n.d.-c).

3.4.2 Otwieranie i zamykanie sesji

Do otwierania oraz zamykania tzw. sesji interaktywnych, pozwalających na wysyłkę faktur, dochodzi gdy aplikacja rejestruje pojawienie się nowej faktury lub faktur do wysyłki w bazie danych. Sesja otwierana jest poprzez przesłanie żądania z zaszyfrowanym kluczem symetrycznym oraz wektorem inicjalizacji do endpointu `/sessions/online`, otrzymując w odpowiedzi identyfikator sesji oraz czas jej ważności. Po wysyłce sesja jest zamykana przez przesłanie żądania do endpointu `/sessions/online/x/close`, gdzie „x” jest zapisanym przy otwarciu identyfikatorem (Ministerstwo Finansów, n.d.-c).

3.4.3 Przesyłanie faktur

Faktury z bazy danych, w formacie XML, są następnie szyfrowane oraz przesyłane do endpointu `/sessions/online/x/invoices`, gdzie „x” to identyfikator sesji interaktywnej. Zwracany jest identyfikator przesyłki, który wykorzystywany jest do sprawdzenia statusu wysyłki. Niepotwierdzone jest jednak czy faktura została przyjęta przez KSeF (Ministerstwo Finansów, n.d.-c).

3.4.4 Sprawdzenie statusu i pobranie Urzędowego Poświadczenia Odbioru

Aplikacja sprawdza status faktury aż do otrzymania odpowiedzi, lub osiągnięcia limitu ponownych żądań ustalonego przez użytkownika w konfiguracji. W przypadku przyjęcia faktury przez KSeF, aplikacja zapisze Urzędowe Poświadczenie Odbioru w bazie danych, a w przypadku odrzucenia oznaczy w bazie danych porażkę. W przypadku otrzymywania odpowiedzi, że faktura dalej jest przetwarzana, oznaczy jej status jako nieznan, oraz podejmie próbę sprawdzenia jej statusu przy

kolejnym cyklu kolejki (Ministerstwo Finansów, n.d.-c).

3.5 Cykl życia aplikacji

W tej sekcji opisane będzie ogólne działanie - od konfiguracji i uruchomienia, przez działanie, po mechanizm bezpiecznego zamknięcia oraz w jaki sposób aplikacja jest wdrażana kontenerowo.

3.5.1 Konfiguracja

Oprogramowanie jest konfigurowane za pomocą pliku `config.yaml`, którego format został wybrany ze względu na swoją prostotę, czytelność dla człowieka oraz powielenie użycia we wdrożeniu kontenerowym (plik `docker-compose.yaml`), któremu dedykowana jest osobna subsekcja.

Struktura pliku konfiguracyjnego przedstawiona jest w tabeli 7.

Tabela 7: Plik konfiguracyjny aplikacji

Parametr	Typ	Opis
ksef.nip	TEXT	Numer Identyfikacji Podatkowej podmiotu, który reprezentuje użytkownik.
ksef.url	TEXT	Oficjalny adres API KSeF, na wypadek zmiany, lub chęci użytkownika do przetestowania oprogramowania na środowisku testowym.
ksef.http_timeout_sec	INTEGER	Maksymalny czas oczekiwania na odpowiedź HTTP z KSeF.
ksef.auth_retry_delay_sec	INTEGER	Przerwa w sekundach pomiędzy kolejnymi próbami uwierzytelnienia w KSeF.
ksef.polling_delay_sec	INTEGER	Przerwa w sekundach pomiędzy kolejnymi zapytaniami o status wysłanej do KSeF faktury.
ksef.confirmation_max_attempts	INTEGER	Maksymalna liczba prób sprawdzenia statusu przesłanej faktury.
sqlite.db_path	TEXT	Ścieżka do lokalnej bazy danych, wybierana przez użytkownika.
sqlite.busy_timeout_ms	INTEGER	Czas oczekiwania SQLite na zwolnienie blokady bazy danych.
server.port	TEXT	Port HTTP/HTTPS, na którym uruchamiana jest aplikacja.

Parametr	Typ	Opis
server.tls_cert_path	TEXT	Ścieżka do certyfikatu TLS wykorzystywanego przez serwer HTTPS.
server.tls_key_path	TEXT	Ścieżka do klucza prywatnego TLS wykorzystywanego przez serwer HTTPS.
user.max_retries	INTEGER	Maksymalna liczba prób ponownych wysyłki faktury lub webhooka w przypadku błędów.
dash_page_size	INTEGER	Liczba faktur wyświetlanych na jednej stronie panelu użytkownika.
polling_interval	INTEGER	Czas w sekundach między skanowaniem bazy danych w poszukiwaniu faktur do wysyłki.
shutdown_timeout_sec	INTEGER	Maksymalny czas oczekiwania na bezpieczne zamknięcie aplikacji.
sender_batch_size	INTEGER	Maksymalna liczba faktur lub webhooków wysyłanych w jednym cyklu działania aplikacji.
sender_worker_limit	INTEGER	Limit równoległych wysyłek faktur lub webhooków wykonywanych naraz przez aplikację.

Źródło: opracowanie własne.

Dodatkowo, użytkownik musi skonfigurować plik `.env` zawierający zmienne środowiskowe, poprzez skopiowanie, zmianę nazwy i uzupełnienie pliku `.env.example`. Te zmienne występują w osobnym pliku, ponieważ wszystkie poza `DASHBOARD_USERNAME` są danymi poufnymi - ich jawność mogłaby umożliwić dostęp do aplikacji osobom trzecim. Zmienne zostały przedstawione w tabeli 8.

Tabela 8: Zmienne środowiskowe wymagane przez aplikację

Zmienna	Opis
KSEF_TOKEN	Token uwierzytelniający uzyskany przez użytkownika z Krajowego Systemu e-Faktur.
APP_API_KEY	Klucz wykorzystywany do uwierzytelniania systemów zewnętrznych korzystających z endpointów API aplikacji.
DASHBOARD_USERNAME	Nazwa użytkownika wymagana podczas uwierzytelniania w panelu użytkownika.

Zmienna	Opis
DASHBOARD_PASSWORD	Hasło wymagane podczas uwierzytelniania w panelu użytkownika.

Źródło: opracowanie własne.

Brak wartości w którymkolwiek z pól nie pozwoli użytkownikowi na uruchomienie aplikacji.

3.5.2 Uruchomienie aplikacji

Po uruchomieniu aplikacji, tworzone są mechanizmy prowadzenia logów, odpowiadające za rejestrowanie wszystkich działań aplikacji. Otwierany jest wcześniej wspomniany plik konfiguracyjny, a jego wartości są walidowane. Wczytywane są również zmienne środowiskowe. Oprogramowanie utworzy odpowiednią bazę danych SQLite, lub spróbuje połączyć się z już istniejącą, jeśli owa istnieje. Następnie tworzony jest specjalny walidator wykorzystujący oficjalny schemat XSD FA(3) do sprawdzenia poprawności strukturalnej faktur.

Tworzony jest klient HTTP oraz struktura aplikacji, z użyciem parametrów z pliku konfiguracyjnego, oraz uruchamiany jest serwer HTTP lub HTTPS (zależnie od tego, czy użytkownik dostarczył certyfikat oraz klucz TLS (Rescorla, 2018)), asynchronicznie z pętlą odpowiadającą za skanowanie bazy danych w poszukiwaniu faktur. Przygotowywany jest również kontekst pozwalający później na bezpieczne zamknięcie aplikacji.

3.5.3 Działanie aplikacji

Aplikacja w ramach swojego działania eksponuje odpowiednie endpointy API HTTP oraz rozpoczyna cykliczne skanowanie bazy danych SQLite. Endpointy służące do komunikacji z zewnętrznymi systemami zwracają ustrukturyzowane odpowiedzi w formacie JSON, a endpointy wspomagające panel użytkownika przekazują HTML lub odpowiedzi specyficzne do HTMX. Eksponowane są endpointy przedstawione w tabeli ??.

Tabela 9: Endpointy eksponowane przez aplikację

Metoda	Endpoint	Dostęp	Opis
POST	/invoices	Bearer	Główny endpoint wejściowy aplikacji. Przyjmuje faktury w formacie JSON i zapisuje je do dalszego przetwarzania.

Metoda	Endpoint	Dostęp	Opis
GET	/invoice?id={id}	Bearer	Endpoint API zwracający status pojedynczej faktury na podstawie wewnętrznego identyfikatora w bazie danych.
GET	/invoice?external_id={external_id}	Bearer	Endpoint API zwracający status pojedynczej faktury na podstawie identyfikatora z systemu zewnętrznego.
DELETE	/deleteinvoice?external_id={external_id}	Bearer	Endpoint API pozwalający usunąć fakturę o statusie FAILED na podstawie identyfikatora zewnętrznego.
GET	/	Publiczny	Endpoint przekierowujący użytkownika do panelu administracyjnego, pod chronionym endpointem /ui/invoices.
GET	/ui/invoices	Basic Auth	Główny widok panelu użytkownika.
GET	/ui/invoicetable	Basic Auth	Endpoint zwracający tabelę faktur, wykorzystywany do odświeżania widoku z użyciem HTMX.
GET	/ui/invoice?id={id}	Basic Auth	Endpoint zwracający szczegółowy widok pojedynczej faktury w panelu użytkownika.
DELETE	/ui/deleteinvoice?id={id}	Basic Auth	Endpoint panelu użytkownika pozwalający ręcznie usunąć fakturę o statusie FAILED.
POST	/ui/retrywebhook?id={id}	Basic Auth	Endpoint panelu użytkownika resetujący licznik prób ponownych webhooka dla wybranej faktury.
GET	/health/live	Publiczny	Endpoint pozwalający sprawdzić, czy aplikacja działa.

Metoda	Endpoint	Dostęp	Opis
GET	/health/ready	Publiczny	Endpoint pozwalający sprawdzić, czy aplikacja jest gotowa do obsługi faktur.

Źródło: opracowanie własne.

Aplikacja działa na dwa równoległe sposoby - zajmuje się obsługą żądań HTTP, wymienionych powyżej, oraz rekordów zapisanych w bazie danych. Istotnym elementem jest sender (pol. „wysyłacz”), który co określony przez użytkownika w trakcie konfiguracji czas przegląda bazę danych w poszukiwaniu nieprzetworzonych faktur, które mogą być oznaczone jako PENDING (pol. oczekujące) lub UNKNOWN (pol. nieznane). W obu przypadkach, są one zbierane, z ogólnym limitem ilości określonym przez użytkownika, po czym zależnie od ich statusu wykonywane są odpowiednie działania. W przypadku faktur oczekujących na przetworzenie, podejmowana jest próba wysyłki oraz potwierdzenia jej statusu. Jeśli wysyłka się nie powiedzie, podejmowane są kolejne próby, do osiągnięcia limitu, gdzie faktura zostanie oznaczona jako FAILED (pol. niepowodzenie). W przypadku, gdy status wysyłki nie zostanie potwierdzony, faktury oznaczane są jako UNKNOWN, a sender zajmie się kolejnymi próbami potwierdzenia ich statusu w następnych cyklach. Na koniec przetwarzania określonego rekordu, jeśli oryginalne żądanie JSON zawierało adres, na który ma być wysłana informacja zwrotna, aplikacja skorzysta z webhooka, by ową informację przesłać. W trakcie każdego cyklu ponawiane są również próby wysłania webhooka, o ile pierwsza próba zakończyła się niepowodzeniem. Wszystkie wysyłki, zarówno faktur, jak i webhooków, odbywają się współbieżnie. Maksymalna ilość procesowanych naraz elementów określana jest przez użytkownika w pliku konfiguracyjnym.

Ważnym elementem architektury programu jest lokalna trwałość danych w bazie SQLite. Przyjęte przez aplikację faktury są przechowywane lokalnie, więc w przypadku niepowodzenia mogą być ponownie przetwarzane, dzięki czemu aplikacja będzie kontynuować operacje pomimo tymczasowych błędów w komunikacji z Krajowym Systemem e-Faktur.

3.5.4 Bezpieczne zamykanie aplikacji

W celu dbania o zgodność danych, oprogramowanie zostało wyposażone w mechanizm bezpiecznego zamknięcia. Aplikacja czeka na sygnał przerywający, po czym przekazuje o nim informację wcześniej wspomnianemu senderowi oraz serwerowi HTTP. Sender następnie oczekuje na zakończenie obecnego cyklu wysyłki, po czym zakańcza swoje działanie. Serwer HTTP czeka, aż zakończy pracę z rozpoczętymi żadaniami HTTP, równocześnie przestając akceptować nowe na swoich endpointach. Gdy wszystkie prace, które były wykonywane gdy otrzymano sygnał zamknięcia zostaną zakończone, aplikacja zamknie się.

3.5.5 Wdrożenie kontenerowe

Kod źródłowy aplikacji zawiera również pliki pozwalające na nieskomplikowane wdrożenie kontenerowe z użyciem Dockera. Budowany kontener zawiera aplikację w postaci pliku wykonywalnego. Dane przechowywane są w zamontowanym folderze z bazą SQLite. Dzięki Docker Compose, uruchomienie aplikacji wymaga od użytkownika jedynie edycji przykładowego pliku konfiguracyjnego poprzez dodanie swoich danych (oraz ewentualną modyfikację pól wpływających na działanie aplikacji) oraz wykonanie pojedynczego polecenia.

3.6 Przykładowy scenariusz użycia aplikacji

3.6.1 Środowisko i cel

Aplikacja została uruchomiona poprzez Docker Compose w celu przeprowadzenia scenariusza testowego obejmującego przyjęcie faktury od zewnętrznego systemu, przetworzenie jej, zapisanie w bazie danych, uwierzytelnienie w środowisku testowym Krajowego Systemu e-Faktur, wysyłkę, pobranie Urzędowego Poświadczenia Odbioru oraz dostarczenie informacji zwrotnej do systemu zewnętrznego. Oprogramowanie komunikowało się ze środowiskiem testowym KSeF <https://api-test.ksef.mf.gov.pl/v2>. Zmienne środowiskowe wykorzystane do uwierzytelnienia nie zostały zaprezentowane.

3.6.2 Uruchomienie aplikacji w ramach scenariusza

W celu uruchomienia aplikacji wywołane zostały dwie komendy: `docker compose up -d --build`, w celu zbudowania i uruchomienia kontenera z aplikacją, oraz `docker compose logs -f app`, w celu śledzenia jej logów. Aplikacja pomyślnie odczytała zmienne środowiskowe, utworzyła bazę danych SQLite, walidator schematu XSD, serwer HTTP i uruchomiła sender.

3.6.3 Scenariusz pomyślnego przetworzenia faktury

Przekazanie faktury do aplikacji

```
1 {
2   "invoice_type": "VAT",
3   "invoice_number": "INVOICE_EXAMPLE",
4   "issue_date": "2026-08-24",
5   "currency": "PLN",
6   "total_amount": 123.00,
7   "seller": {
8     "nip": "1141414141",
9     "name": "Sprzedawca testowy",
```

```

10     "address_line_1": "ul. Grodzka 50, Kraków"
11 },
12 "buyer": {
13     "nip": "6793199607",
14     "name": "Nabywca testowy",
15     "address_line_1": "ul. Krakowska 16/4, Kraków"
16 },
17 "items": [
18     {
19         "line_number": 1,
20         "name": "Produkt testowy",
21         "unit": "szt.",
22         "quantity": 1,
23         "unit_price_net": 100.00,
24         "net_amount": 100.00,
25         "tax_rate": "23"
26     }
27 ],
28 "tax_breakdowns": [
29     {
30         "tax_rate": "23",
31         "net_amount": 100.00,
32         "tax_amount": 23.00
33     }
34 ],
35 "callback_url": "http://webhook:8080/webhook"
36 }

```

Listing 9: Dane faktury zawierającej uprawniony NIP sprzedawcy

Następnie do aplikacji przesłane zostało żądanie HTTP zawierające fakturę testową, za pomocą programu curl, następującą komendą:

```

1 curl -X POST http://localhost:8080/invoices \
2   -H "Authorization: Bearer [APP_API_KEY]" \
3   -H "Content-Type: application/json" \
4   --data-binary @invoice.json

```

Listing 10: Przekazanie faktury do aplikacji

@invoice.json odwołuje się do pliku w formacie JSON zawierającego dane z listingu 9.

W odpowiedzi aplikacja zwróciła kod HTTP 201 oraz następujące dane w formacie JSON:

```

1 {
2   "status": "ok",
3   "message": "the invoice has been created.",
4   "data": {
5     "id": 1,
6     "status": "PENDING"
7   }
8 }

```

Listing 11: Odpowiedź aplikacji po przyjęciu faktury

Ta odpowiedź potwierdza przyjęcie faktury przez aplikację, walidację jej danych, transformację do formatu XML, walidację wobec schematu XSD oraz zapis w bazie danych.

Przetworzenie faktury przez aplikację

```

1 [INFO] 14:06:50 event=invoice_received external_id="INVOICE_EXAMPLE"
2 [INFO] 14:06:50 event=invoice_created invoice_id=1 external_id="INVOICE_EXAMPLE
   " status="PENDING"
3 [INFO] 14:07:01 event=session_opened kind="pending" session_ref="20260824-SO
   -3077B12000-B3DCA8DCEE-73" count=1
4 [INFO] 14:07:01 event=invoice_send_started invoice_id=1 external_id="
   INVOICE_EXAMPLE" session_ref="20260824-SO-3077B12000-B3DCA8DCEE-73"
5 [INFO] 14:07:02 event=invoice_submission_created invoice_id=1 external_id="
   INVOICE_EXAMPLE" session_reference="20260824-SO-3077B12000-B3DCA8DCEE-73"
   submission_reference="20260824-EE-3077BB1000-B3DF56A810-A1"
6 [INFO] 14:07:02 event=invoice_confirmation_started invoice_id=1 external_id="
   INVOICE_EXAMPLE" submission_reference="20260824-EE-3077BB1000-B3DF56A810-A1
   " session_ref="20260824-SO-3077B12000-B3DCA8DCEE-73"
7 [INFO] 14:07:07 event=upo_downloaded invoice_id=1 external_id="INVOICE_EXAMPLE"
   bytes=5476
8 [INFO] 14:07:07 event=invoice_sent invoice_id=1 external_id="INVOICE_EXAMPLE"
   ksef_id="1141414141-20260824-71537C800000-C0" submission_reference
   ="20260824-EE-3077BB1000-B3DF56A810-A1"
9 [INFO] 14:07:07 event=webhook_delivered invoice_id=1 external_id="
   INVOICE_EXAMPLE"
10 [INFO] 14:07:07 event=session_closed kind="pending" session_ref="20260824-SO
   -3077B12000-B3DCA8DCEE-73"

```

Listing 12: Logi przetwarzania faktury testowej

Wyżej zamieszczone logi przedstawiają proces przetwarzania testowej faktury przez oprogramowanie. Pierwsze zdarzenie, oznaczone jako event=invoice_created, wskazuje,

że faktura została zaakceptowana przez aplikację, przetransformowana do formatu XML oraz zwalidowana wobec schematu XSD. Następne zdarzenie, `event=session_opened`, to otwarcie sesji interaktywnej po pomyślnym uwierzytelnieniu w Krajowym Systemie e-Faktur, w celu wysyłki wcześniej wspomnianej faktury. Faktura jest następnie wysyłana, a identyfikator wysyłki (`submission_reference=20260824-EE-3077BB1000-B3DF56A810-A1`) zapisywany, w celu późniejszej weryfikacji statusu akceptacji przez KSeF. Rozpoczyna się weryfikacja pomyślności wysyłki poprzez wysłanie zapytania z identyfikatorem, która skutkuje pobraniem Urzędowego Poświadczenia Odbioru. Faktura oznaczana jest jako wysłana, a jej identyfikator nadany przez KSeF (`ksef_id="1141414141-20260824-71537C800000-C0"`) zostaje zapisany, po czym wysyłany jest webhook do systemu zewnętrznego, na co wskazuje `event=webhook_delivered`. Po zakończeniu przetwarzania, sesja interaktywna jest zamykana, a sender wraca do cyklicznej pracy skanowania bazy danych w poszukiwaniu rekordów do przetworzenia.

Weryfikacja rezultatu

```
1 {
2   "event": "invoice.accepted",
3   "invoice_id": 1,
4   "external_id": "INVOICE_EXAMPLE",
5   "status": "SENT",
6   "ksef_reference_number": "1141414141-20260824-71537C800000-C0",
7   "submission_reference": "20260824-EE-3077BB1000-B3DF56A810-A1",
8   "error_message": null,
9   "timestamp": "2026-08-24T14:07:07Z"
10 }
```

Listing 13: Webhook wysłany do systemu zewnętrznego

```
1 curl -G http://localhost:8080/invoice \
2   -H "Authorization: Bearer [APP_API_KEY]" \
3   --data-urlencode "external_id=INVOICE_EXAMPLE"
```

Listing 14: Zapytanie o końcowy status faktury

```
1 {
2   "status": "ok",
3   "message": "invoice status received.",
4   "data": {
5     "id": 1,
6     "external_id": "INVOICE_EXAMPLE",
7     "status": "SENT",
```

```

8   "ksef_id": "1141414141-20260824-71537C800000-C0",
9   "ksef_error": null,
10  "submission_reference": "20260824-EE-3077BB1000-B3DF56A810-A1",
11  "attempt_count": 0,
12  "created_at": "2026-08-24T14:06:50Z",
13  "updated_at": "2026-08-24T14:07:07.736397823Z",
14  "webhook_delivered": true,
15  "webhook_error": null
16 }
17 }

```

Listing 15: Końcowy status faktury zwrócony przez aplikację

Powyżej pokazane zostały rezultaty przetwarzania faktury. Listing 13 przedstawia treść informacji zwrotnej przesłanej do systemu zewnętrznego. Następne dwa listingi, 14 oraz 15, przedstawiają zapytanie o status faktury przesłane przez system zewnętrzny do oprogramowania za pomocą programu `curl` oraz odpowiedź z danymi. Status `SENT` wskazuje na pomyślną wysyłkę, `"attempt_count": 0` na brak prób ponownych wysyłki, a `"webhook_delivered": true` na przesłanie informacji zwrotnej do systemu zewnętrznego.

3.6.4 Scenariusz odrzucenia faktury przez KSeF

Założenia i lokalne przyjęcie faktury

```

1  {
2    "invoice_type": "VAT",
3    "invoice_number": "FAILURE_EXAMPLE",
4    "issue_date": "2026-08-24",
5    "currency": "PLN",
6    "total_amount": 123.00,
7    "seller": {
8      "nip": "6793194113",
9      "name": "Sprzedawca testowy",
10     "address_line_1": "ul. Grodzka 50, Kraków"
11   },
12   "buyer": {
13     "nip": "6793199607",
14     "name": "Nabywca testowy",
15     "address_line_1": "ul. Krakowska 16/4, Kraków"
16   },
17   "items": [
18     {
19       "line_number": 1,

```

```

20     "name": "Produkt testowy",
21     "unit": "szt.",
22     "quantity": 1,
23     "unit_price_net": 100.00,
24     "net_amount": 100.00,
25     "tax_rate": "23"
26   }
27 ],
28 "tax_breakdowns": [
29   {
30     "tax_rate": "23",
31     "net_amount": 100.00,
32     "tax_amount": 23.00
33   }
34 ],
35 "callback_url": "http://webhook:8080/webhook"
36 }

```

Listing 16: Dane faktury zawierającej nieuprawniony NIP sprzedawcy

Powyższa faktura została przesłana do aplikacji w ten sam sposób co faktura z pomyślnego scenariusza, poprzez zastosowanie programu curl:

```

1 curl -X POST http://localhost:8080/invoices \
2   -H "Authorization: Bearer [APP_API_KEY]" \
3   -H "Content-Type: application/json" \
4   --data-binary @invoice_failure.json

```

Listing 17: Przekazanie faktury z nieuprawnionym NIP sprzedawcy do aplikacji

```

1 {
2   "status": "ok",
3   "message": "the invoice has been created.",
4   "data": {
5     "id": 2,
6     "status": "PENDING"
7   }
8 }

```

Listing 18: Odpowiedź aplikacji po przyjęciu faktury

Aplikacja zaakceptowała fakturę, przetransformowała ją do formatu XML oraz zwalidowała wobec schematu XSD.

Odrzucenie faktury


```

1 [INFO] 14:54:02 event=invoice_received external_id="FAILURE_EXAMPLE"
2 [INFO] 14:54:02 event=invoice_created invoice_id=2 external_id="FAILURE_EXAMPLE
  " status="PENDING"
3 [INFO] 14:54:10 event=session_opened kind="pending" session_ref="20260824-SO
  -332A531000-E0A781FEEA-6D" count=1
4 [INFO] 14:54:10 event=invoice_send_started invoice_id=2 external_id="
  FAILURE_EXAMPLE" session_ref="20260824-SO-332A531000-E0A781FEEA-6D"
5 [INFO] 14:54:10 event=invoice_submission_created invoice_id=2 external_id="
  FAILURE_EXAMPLE" session_reference="20260824-SO-332A531000-E0A781FEEA-6D"
  submission_reference="20260824-EE-332A5C9000-C6CA6EB2C2-38"
6 [INFO] 14:54:10 event=invoice_confirmation_started invoice_id=2 external_id="
  FAILURE_EXAMPLE" submission_reference="20260824-EE-332A5C9000-C6CA6EB2C2
  -38" session_ref="20260824-SO-332A531000-E0A781FEEA-6D"
7 [ERROR] 14:54:16 event=invoice_confirmation_rejected invoice_id=2 external_id="
  FAILURE_EXAMPLE" error="KSeF rejected the invoice: code=410, description=
  Nieprawidłowy zakres uprawnień, extensions=map[], details=[Kontekst
  1141414141 nie jest uprawniony do wystawienia faktury w imieniu sprzedawcy
  (NIP: 6793194113)]"
8 [INFO] 14:54:16 event=webhook_delivered invoice_id=2 external_id="
  FAILURE_EXAMPLE"
9 [INFO] 14:54:16 event=session_closed kind="pending" session_ref="20260824-SO
  -332A531000-E0A781FEEA-6D"

```

Listing 19: Logi odrzucenia faktury przez KSeF

Zdarzenia przedstawione w logach są niemal identyczne do tych w pomyślnym scenariuszu, aż do zdarzenia `event=invoice_confirmation_rejected`, wskazującego, że faktura została odrzucona. Powód odrzucenia znajduje się w polu `error`, gdzie wartością jest zwrócony przez Krajowy System e-Faktur błąd. Token z Panelu Podatnika podany przez użytkownika uprawnia go do wystawiania faktur w imieniu podmiotu o Numerze Identyfikacji Podatkowej 1141414141, a przesłana faktura zakładała sprzedawcę o NIP 6793194113. Następnie przesłany został webhook, a sesja interaktywna została zamknięta.

Weryfikacja rezultatu

```

1 {
2   "event": "invoice.failed",
3   "invoice_id": 2,
4   "external_id": "FAILURE_EXAMPLE",
5   "status": "FAILED",
6   "ksef_reference_number": null,
7   "submission_reference": "20260824-EE-332A5C9000-C6CA6EB2C2-38",

```

```

8  "error_message": "KSeF rejected the invoice: code=410, description=Nieprawidł
    owy zakres uprawnień, extensions=map[], details=[Kontekst 1141414141 nie
    jest uprawniony do wystawienia faktury w imieniu sprzedawcy (NIP:
    6793194113)]",
9  "timestamp": "2026-08-24T14:54:16.227674492Z"
10 }

```

Listing 20: Webhook informujący o odrzuceniu faktury

```

1  curl -G http://localhost:8080/invoice \
2  -H "Authorization: Bearer [APP_API_KEY]" \
3  --data-urlencode "external_id=FAILURE_EXAMPLE"

```

Listing 21: Zapytanie o końcowy status odrzuconej faktury

```

1  {
2  "status": "ok",
3  "message": "invoice status received.",
4  "data": {
5    "id": 2,
6    "external_id": "FAILURE_EXAMPLE",
7    "status": "FAILED",
8    "ksef_id": null,
9    "ksef_error": "KSeF rejected the invoice: code=410, description=Nieprawidł
    owy zakres uprawnień, extensions=map[], details=[Kontekst 1141414141 nie
    jest uprawniony do wystawienia faktury w imieniu sprzedawcy (NIP:
    6793194113)]",
10   "submission_reference": "20260824-EE-332A5C9000-C6CA6EB2C2-38",
11   "attempt_count": 0,
12   "created_at": "2026-08-24T14:54:02Z",
13   "updated_at": "2026-08-24T14:54:16.231871938Z",
14   "webhook_delivered": true,
15   "webhook_error": null
16  }
17 }

```

Listing 22: Końcowy status odrzuconej faktury

Na powyższych przykładach przedstawione zostały rezultaty przetwarzania faktury - treść informacji zwrotnej przesłanej do systemu zewnętrznego, gdzie FAILED wskazuje na odrzucenie faktury, a error_message przekazuje błąd zwrócony przez KSeF oraz odpowiedź na zapytanie przesłane w listingu 21 w celu weryfikacji statusu faktury.

3.7 Ograniczenia aplikacji

Największym ograniczeniem aplikacji jest brak wsparcia dla trybów offline24/offline (Ministerstwo Finansów, 2025e, 2025f) - oprogramowanie przewiduje tylko tymczasowe przerwy w działaniu Krajowego Systemu e-Faktur, dlatego też wyposażone jest w mechanizm lokalnej trwałości danych (baza danych, ponowne próby wysyłki).

Drugim głównym ograniczeniem jest brak pełnego pokrycia dla schematu FA(3), który przewiduje scenariusze takie jak dostawa nowych środków transportu lub udział trzeciego podmiotu w transakcji (Ministerstwo Finansów, 2025a). Aplikacja przewiduje wykorzystanie jej dla małych i średnich przedsiębiorstw.

Na poniższej liście przedstawione są inne, pomniejsze ograniczenia:

- wsparcie dla pojedynczego podmiotu,
- dane są przechowywane lokalnie bez enkrypcji,
- aplikacja z góry ufa adresom systemów zewnętrznych podanym wraz z danymi faktury.

Zakończenie

Celem pracy było utworzenie aplikacji pośredniczącej między zewnętrznymi systemami, a Krajowym Systemem e-Faktur, ze szczególnym naciskiem na prostotę wdrożenia oraz obsługi.

Aplikacja została utworzona z wykorzystaniem języka Go, z bazą danych SQLite oraz panelem użytkownika wykorzystującym HTMX. Wykorzystane technologie pozwoliły na spełnienie większości założeń. Cel pracy został osiągnięty, jak zostało zademonstrowane na końcu rozdziału trzeciego, w dwóch scenariuszach: jednym prezentującym przykład poprawnie przetworzonej faktury i jednym z fakturą odrzuconą przez KSeF. W wyniku pracy powstał pojedynczy plik wykonywalny.

Możliwe byłoby rozszerzenie funkcjonalności aplikacji, przede wszystkim poprzez wyeliminowanie ograniczeń wyliczonych na końcu rozdziału trzeciego: wsparcie dla trybów offline oraz offline24, wsparcie dla wielu podmiotów, czy poszerzenie pokrycia schematu FA(3).

Pełne repozytorium z aplikacją można znaleźć pod adresem <https://github.com/brunonsocha/ksef-integration>.

Bibliografia

1. sqlite package - modernc.org/sqlite - Go Packages. (n.d.). Retrieved January 16, 2026, from <https://pkg.go.dev/modernc.org/sqlite>
2. Big Sky Software. (n.d.). htmx Documentation. Retrieved August 29, 2026, from <https://htmx.org/docs/>
3. Bray, T. (2017, December). The JavaScript Object Notation (JSON) Data Interchange Format. <https://doi.org/10.17487/RFC8259>
4. Council of the European Union. (2025, March). Council Directive (EU) 2025/516 of 11 March 2025 amending Directive 2006/112/EC as regards VAT rules for the digital age. Retrieved January 16, 2026, from <http://data.europa.eu/eli/dir/2025/516/oj>
5. Docker, Inc. (n.d.-a). Docker overview. Retrieved August 29, 2026, from <https://docs.docker.com/get-started/docker-overview/>
6. Docker, Inc. (n.d.-b). What is a container? Retrieved August 29, 2026, from <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>
7. European Commission, CASE, Oxford Economics, & Syntesia. (2025). VAT gap in the EU – Country report 2024 – Poland. <https://doi.org/10.2778/2516273>
8. European Parliament & Council of the European Union. (2014, April). Directive 2014/55/EU of the European Parliament and of the Council of 16 April 2014 on electronic invoicing in public procurement Text with EEA relevance. Retrieved January 16, 2026, from <http://data.europa.eu/eli/dir/2014/55/oj>
9. Fielding, R. T., Nottingham, M., & Reschke, J. (2022, June). HTTP Semantics. <https://doi.org/10.17487/RFC9110>
10. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Doctoral dissertation, University of California, Irvine]. Retrieved August 29, 2026, from <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
11. Google. (n.d.-a). The Go Programming Language Specification - The Go Programming Language. Retrieved January 16, 2026, from <https://go.dev/ref/spec>

12. Google. (n.d.-b). Standard library - Go Packages. Retrieved January 16, 2026, from <https://pkg.go.dev/std>
13. Goshima, M. (n.d.). yaml package - github.com/goccy/go-yaml - Go Packages. Retrieved January 16, 2026, from <https://pkg.go.dev/github.com/goccy/go-yaml#section-readme>
14. International Organization for Standardization. (n.d.). ISO 4217 – Currency codes. Retrieved August 29, 2026, from <https://www.iso.org/iso-4217-currency-codes.html>
15. Jones, M. B., & Hardt, D. (2012, October). The OAuth 2.0 Authorization Framework: Bearer Token Usage. <https://doi.org/10.17487/RFC6750>
16. Mazur, T., Bach, D., Juźwik, A., Czechowicz, I., & Bieńkowska, J. (2019). Raport na temat wielkości luki podatkowej w podatku VAT w Polsce w latach 2004-2017. Retrieved August 29, 2026, from <https://www.gov.pl/web/finanse/no-3-2019>
17. Ministerstwo Finansów. (2025a, June). Faktura ustrukturyzowana i struktura logiczna FA - KSeF (Krajowy System e-Faktur). Retrieved January 16, 2026, from <https://ksef.podatki.gov.pl/informacje-ogolne-ksef-20/faktura-ustrukturyzowana-i-struktura-logiczna-fa/>
18. Ministerstwo Finansów. (2025b). JPK_VAT z deklaracją. Retrieved August 29, 2026, from https://www.podatki.gov.pl/podatki-firmowe/jednolity-plik-kontrolny/jpk_vat-z-deklaracja/jpk_vat-z-deklaracja
19. Ministerstwo Finansów. (2025c, June). Podstawy Prawne KSeF - KSeF (Krajowy System e-Faktur). Retrieved January 6, 2026, from <https://ksef.podatki.gov.pl/informacje-ogolne-ksef-20/podstawy-prawne-oraz-kluczowe-terminy>
20. Ministerstwo Finansów. (2025d, September). Pytania i odpowiedzi KSeF 2.0 - KSeF (Krajowy System e-Faktur). Retrieved January 6, 2026, from <https://ksef.podatki.gov.pl/pytania-i-odpowiedzi-ksef-20>
21. Ministerstwo Finansów. (2025e, June). Tryb offline - niedostępność KSeF - KSeF (Krajowy System e-Faktur). Retrieved January 16, 2026, from <https://ksef.podatki.gov.pl/informacje-ogolne-ksef-20/tryb-offline-niedostepnosc-ksef/>
22. Ministerstwo Finansów. (2025f, June). Tryb offline24 - KSeF (Krajowy System e-Faktur). Retrieved January 16, 2026, from <https://ksef.podatki.gov.pl/informacje-ogolne-ksef-20/tryb-offline24/>
23. Ministerstwo Finansów. (2025g, June). Zakres obowiązkowego KSeF - KSeF (Krajowy System e-Faktur). Retrieved January 6, 2026, from <https://ksef.podatki.gov.pl/informacje-ogolne-ksef-20/zakres-obowiazkowego-ksef>

24. Ministerstwo Finansów. (2026). Broszura informacyjna dotycząca struktury logicznej FA(3). Retrieved August 29, 2026, from <https://ksef.podatki.gov.pl/media/jknpcymf/broszura-informacyjna-dotyczaca-struktury-logicznej-fa-3-04032026.pdf>
25. Ministerstwo Finansów. (n.d.-a). Informacja dotycząca metody liczenia luki VAT - Ministerstwo Finansów - Portal Gov.pl. Retrieved January 15, 2026, from <https://www.gov.pl/web/finanse/informacja-dotyczaca-metody-liczenia-luki-vat>
26. Ministerstwo Finansów. (n.d.-b). Jednolity Plik Kontrolny - najważniejsze informacje - Informacje prasowe - Dla mediów - Ministerstwo Finansów. Retrieved January 16, 2026, from https://mf-arch2.mf.gov.pl/web/bip/ministerstwo-finansow/dla-mediow/informacje-prasowe/-/asset_publisher/6PxF/content/jednolity-plik-kontrolny-najwazniejsze-informacje?redirect=https%3A%2F%2Fmf-arch2.mf.gov.pl%2Fweb%2Fbip%2Fministerstwo-finansow%2Fdla-mediow%2Finformacje-prasowe%3Fp_p_id%3D101_INSTANCE_6PxF%26p_p_lifecycle%3D0%26p_p_state%3Dnormal%26p_p_mode%3Dview%26p_p_col_id%3Dcolumn-2%26p_p_col_count%3D1%26p_r_p_564233524_tag%3Djednolity%2Bplik%2Bkontrolny%26_101_INSTANCE_6PxF_changeViewTo%3Dabstracts
27. Ministerstwo Finansów. (n.d.-c). KSeF API 2.0 – dokumentacja środowiska testowego. Retrieved August 29, 2026, from <https://api-test.ksef.mf.gov.pl/docs/v2/>
28. Red Hat. (2024, February). What is a webhook? Retrieved September 1, 2026, from <https://www.redhat.com/en/topics/automation/what-is-a-webhook>
29. Reschke, J. (2015, September). The Basic HTTP Authentication Scheme. <https://doi.org/10.17487/RFC7617>
30. Rescorla, E. (2018, August). The Transport Layer Security (TLS) Protocol Version 1.3. <https://doi.org/10.17487/RFC8446>
31. SQLite Development Team. (n.d.-a). About SQLite. Retrieved August 29, 2026, from <https://www.sqlite.org/about.html>
32. SQLite Development Team. (n.d.-b). PRAGMA Statements. Retrieved August 29, 2026, from <https://www.sqlite.org/pragma.html>
33. Tailwind Labs. (n.d.). Styling with utility classes. Retrieved August 29, 2026, from <https://tailwindcss.com/docs/styling-with-utility-classes>
34. The Go Authors. (n.d.). Frequently Asked Questions (FAQ). Retrieved August 29, 2026, from <https://go.dev/doc/faq>
35. The Helium Authors. (2026). helium package – github.com/lestrrat-go/helium v0.8.0. Retrieved August 29, 2026, from <https://pkg.go.dev/github.com/lestrrat-go/helium@v0.8.0>

36. World Wide Web Consortium. (2008, November). Extensible Markup Language (XML) 1.0 (Fifth Edition). Retrieved August 29, 2026, from <https://www.w3.org/TR/xml/>
37. World Wide Web Consortium. (2012, April). W3C XML Schema Definition Language (XSD) 1.1 Part 1: Structures. Retrieved August 29, 2026, from <https://www.w3.org/TR/xmlschema11-1/>

Spis tabel

1	Wielkość luki VAT w Polsce w latach 2004–2023	7
2	Biblioteki zewnętrzne wykorzystane w projekcie	12
3	Najważniejsze kolumny tabeli Invoices w lokalnej bazie danych	29
4	Wartości kolumny status w tabeli Invoices	30
5	Pola żądania wejściowego HTTP przekazywanego do endpointu /invoices . .	33
6	Pola żądania HTTP z informacją zwrotną przesyłanego do systemu zewnętrznego	36
7	Plik konfiguracyjny aplikacji	38
8	Zmienne środowiskowe wymagane przez aplikację	39
9	Endpointy eksponowane przez aplikację	40

Spis rysunków

1	Architektura aplikacji integracyjnej	28
2	Przejścia pomiędzy stanami przetwarzania faktury	32

Spis listingów

1	Przykład współbieżnego przetwarzania faktur	13
2	Przykład mechanizmu bezpiecznego zamykania aplikacji	14
3	Przykład połączenia z bazą SQLite	15
4	Przykładowa faktura w formacie JSON odbierana od systemu zewnętrznego . .	18
5	Przykładowa informacja zwrotna (webhook) w formacie JSON przesyłana do systemu zewnętrznego po poprawnym przetworzeniu faktury	19
6	Przykład endpointu zwracającego fragment HTML dla HTMX	21
7	Przykład użycia atrybutów HTMX dla widoku szczegółów faktury	22
8	Przykład użycia klas Tailwind CSS w przycisku panelu użytkownika	23
9	Dane faktury zawierającej uprawniony NIP sprzedawcy	43
10	Przekazanie faktury do aplikacji	44
11	Odpowiedź aplikacji po przyjęciu faktury	44
12	Logi przetwarzania faktury testowej	45
13	Webhook wysłany do systemu zewnętrznego	46
14	Zapytanie o końcowy status faktury	46
15	Końcowy status faktury zwrócony przez aplikację	46
16	Dane faktury zawierającej nieuprawniony NIP sprzedawcy	47
17	Przekazanie faktury z nieuprawnionym NIP sprzedawcy do aplikacji	48
18	Odpowiedź aplikacji po przyjęciu faktury	48
19	Logi odrzucenia faktury przez KSeF	48
20	Webhook informujący o odrzuceniu faktury	49
21	Zapytanie o końcowy status odrzuconej faktury	49
22	Końcowy status odrzuconej faktury	50